

Discipline of Discovery

Experiment Ledgers, Validation Strategies & Decision Integrity

This session explains how to track your progress and test your models. We focus on being honest with data to make reliable business decisions.

The "Lab Notebook" Mentality

Why a "Process" Needs a "Notebook"?

Last session, we discussed the "ML Lifecycle" as a process. But a process is just a map. To move forward, you need a Lab Notebook (The Ledger).

Old Way: "I tried some parameters and found a good result." (Random)

New Way: "I tested 10 hypotheses, recorded every failure, and identified the best path." (Scientific)



The "Lab Notebook" Mentality

Stop Guessing, Start Measuring.

A Lab Notebook is an immutable record of your thoughts, actions, and results. If it is not in the Ledger, the experiment never happened.



The "Lab Notebook" Mentality

Why do we do this?

1. Reproducibility: Can you get the same result tomorrow?
2. Anti-Self-Deception: It is easy to remember only the "good" results. The Ledger remembers the truth.
3. Decision Defense: Your project grade (or your promotion at work) depends on your reasoning, not just your accuracy score.



The "Lab Notebook" Mentality

Why do we do this?

1. Reproducibility: Can you get the same result tomorrow?
2. Anti-Self-Deception: It is easy to remember only the "good" results. The Ledger remembers the truth.
3. Decision Defense: Your project grade (or your promotion at work) depends on your reasoning, not just your accuracy score.



The "Lab Notebook" Mentality

What to Record?

The Inputs (The Setup):

1. Data Version: Which subset of data did you use?
2. Hyperparameters: Learning rate, number of trees, etc.
3. Seed Value: For 100% reproducibility.



The "Lab Notebook" Mentality

What to Record?

The Process (The "Why"):

1. "Testing if removing 'Age' feature improves precision."

The Outputs (The Evidence):

1. Primary Metric: (e.g., MSE or Accuracy).
2. Validation Evidence: (e.g., Confusion Matrix snapshot).



The "Lab Notebook" Mentality

A perfect score without a Ledger is a failure.

A low score with a perfect Ledger is an engineering success



The "Lab Notebook" Mentality

A perfect score without a Ledger is a failure.

A low score with a perfect Ledger is an engineering success



The "Lab Notebook" Mentality

The simplest approach for you is using **GitHub** for code (and other assets) and a (cloud based, versioned, backed up) **Excel** file for history.



The "Lab Notebook" Mentality

GitHub (The Archive): Every time you change your model, commit your code. Use descriptive commit messages (e.g., "Added Winter's Method with Seasonality=12").

Excel (The Lab Ledger): * This is your Journal. After every GitHub commit, record the results in your Excel table.

Your Excel sheet should include the Commit Hash (the ID of your code version) so you can go back in time.



The "Lab Notebook" Mentality

Minimum columns for your Excel Ledger.

- Experiment ID: #001, #002...
- Commit Hash: (First 7 characters of your GitHub commit).
- Hypothesis: "Does adding 'Rain' data improve prediction?"
- Parameters: $k=5$, $\alpha=0.2$, $\gamma=0.5$.
- Validation Metrics: Precision, Recall, or MSE. (All with values, not what you measure)



The "Lab Notebook" Mentality

Minimum columns for your Excel Ledger.
(cnt'd)

- Observation: "Model is better at predicting rain, but worse at sunny days."

Here "code" refers to the whole file set involved in the experiment. So different versions of data files and a previously saved trained model file that has been loaded also is in this set.

Don't just record successes. Recording a failure (e.g., 'Model crashed') is more



The "Lab Notebook" Mentality

Don't just record successes. Recording a failure (e.g., 'Model crashed') is more important for your final defense.

Someone could pick a random row from your Excel and say: 'Show me the code for Experiment #04

The Github hashes create a time machine.



The "Lab Notebook" Mentality

Don't just record successes. Recording a failure (e.g., 'Model crashed') is more important for your final defense.

Someone could pick a random row from your Excel and say: 'Show me the code for Experiment #04

The Github hashes create a time machine.



The "Lab Notebook" Mentality

How to work like a Data Scientist.

Step 1: Change one thing in your Python code (e.g., change a hyperparameter).

Step 2: Run the experiment.

Step 3: Save your code to GitHub (Git Commit).

Step 4: Open your Excel Ledger. Write down the Commit Hash and the Result.

Step 5: Look at the Mirror. Is the new model better? Why?



The "Lab Notebook" Mentality

Some key terms in this mentality are also important in your discussion (ie. Speak like a Data Scientist as well).



The "Lab Notebook" Mentality

Immutability (Unchangeability)

- What: Once a version is saved, it should never be changed.
- Why: If you change the code of "Experiment #5" later, you lose the truth of that experiment.
- How: In our setup, every GitHub Commit is immutable. Each commit has a unique ID (Hash) that represents that exact moment.



The "Lab Notebook" Mentality

Traceability (The Paper Trail)

- What: The ability to connect a result back to the code and data that created it.
- Why: If a model performs great, you must be able to prove exactly how you built it.
- How: We link the Commit Hash in GitHub to the Result Row in Excel. This creates a bridge between code and performance.



The "Lab Notebook" Mentality

Reproducibility (The Copy-Paste Test)

- What: If I give your code to someone else, they should get the exact same result.
- Why: ML involves randomness. Without reproducibility, your "90% accuracy" might just be luck.
- How: We record Random Seeds and Hyperparameters in the Ledger.



The "Lab Notebook" Mentality

Transparency (No Black Box)

- What: Every decision (e.g., "Why did I delete these rows?") is documented.
- Why: In business, "I don't know why the model did that" is an unacceptable answer.
- How: We use the "Notes/Observation" column in the Excel Ledger to explain our logic.



The "Lab Notebook" Mentality

Accountability (The Decision Defense)

- What: Taking responsibility for the model's errors (FP/FN).
- Why: You are the architect. You must explain why the model fails in certain cases.
- How: By analyzing the Confusion Matrix or Residuals and recording the findings in the Ledger.



The "Lab Notebook" Mentality

Versioning (Time Travel)

- What: Keeping track of different stages of your project.
- Why: Sometimes the "new" model is worse than the "old" one. You need to be able to go back.
- How: Each row in your Excel represents a different version of your ML/AI strategy.

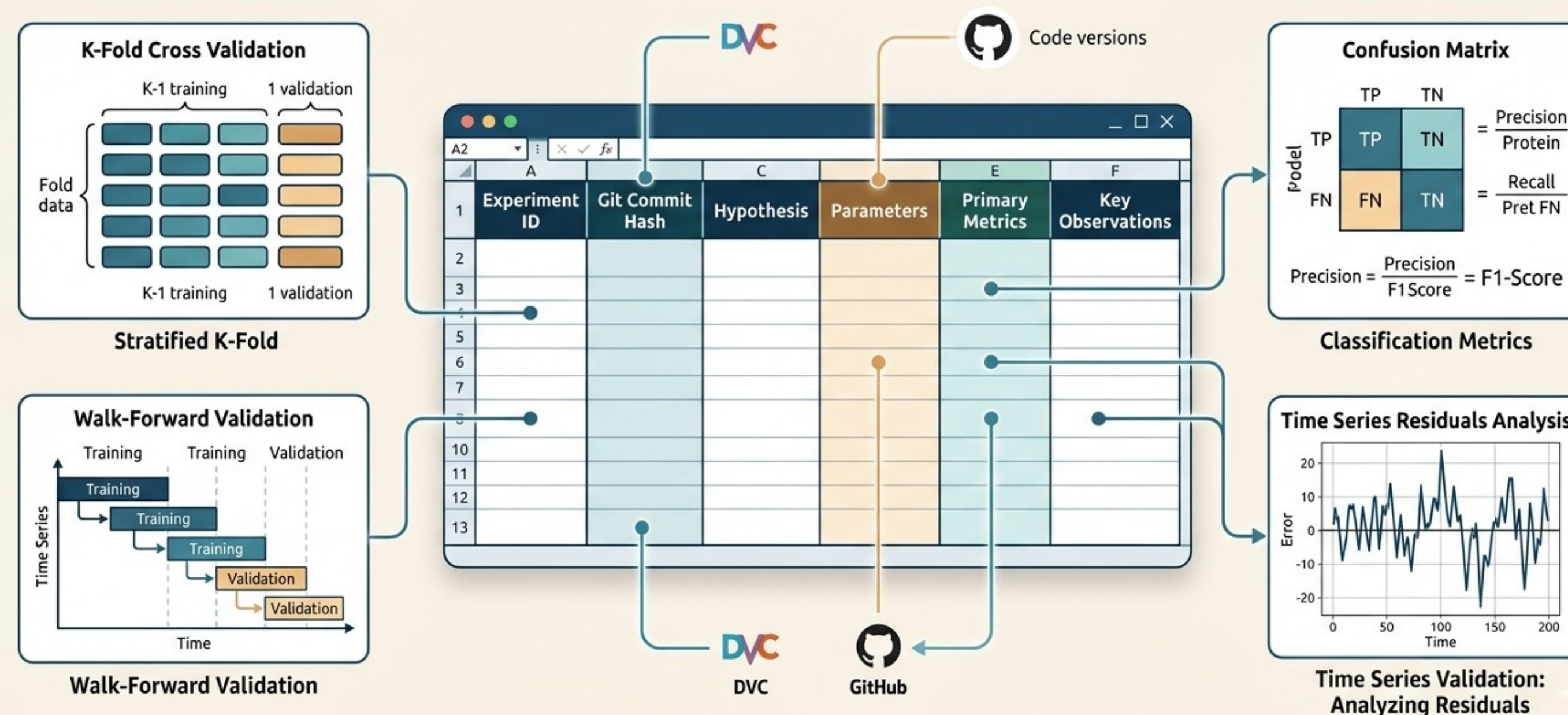


The Anatomy of an Experiment Ledger

If I delete your Python code on your computer but keep your Excel Ledger and your GitHub History, can I rebuild your entire project? If the answer is YES, your Anatomy is perfect.

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY

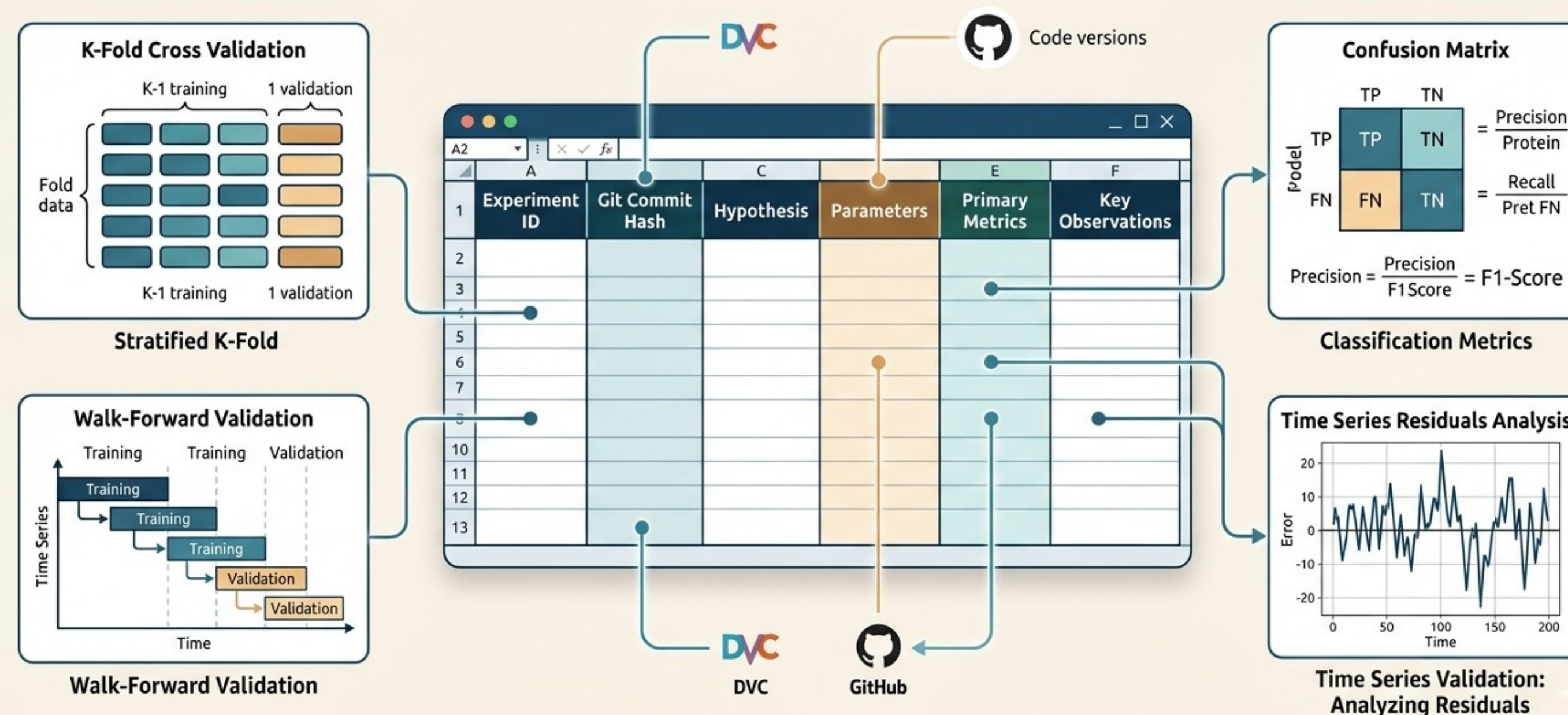


The Anatomy of an Experiment Ledger

Ledger Anatomy for Classification: We should be able to track classifiers beyond score metrics and focus on error maps.

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY



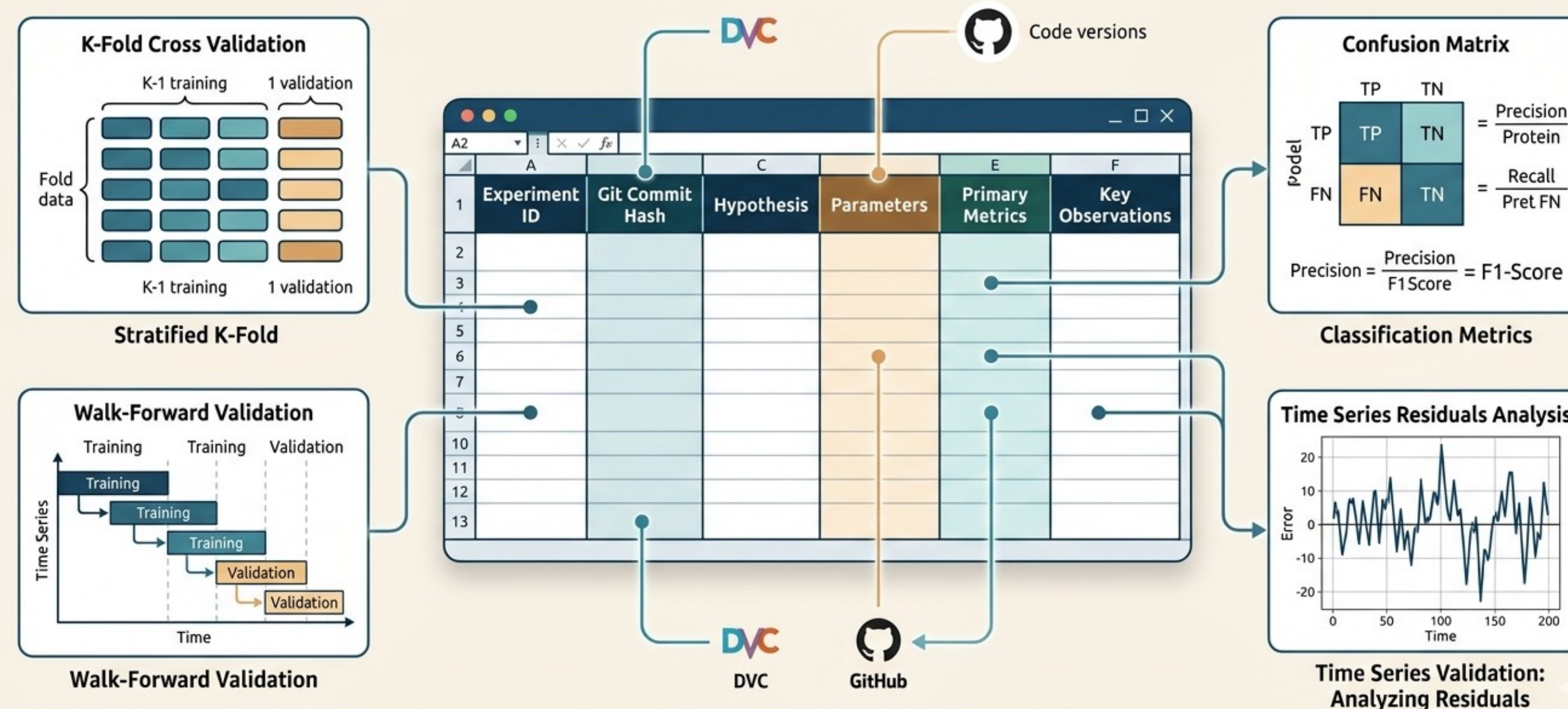
The Anatomy of an Experiment Ledger

Key Inputs to Record:

- Algorithm: Logistic Regression, Random Forest, etc.
- Decision Threshold: (e.g., Default is 0.5, but did you change it?)
- Class Weights: How did you handle imbalanced data?

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY



The Anatomy of an Experiment Ledger

Key Outputs (The Ledger Columns):

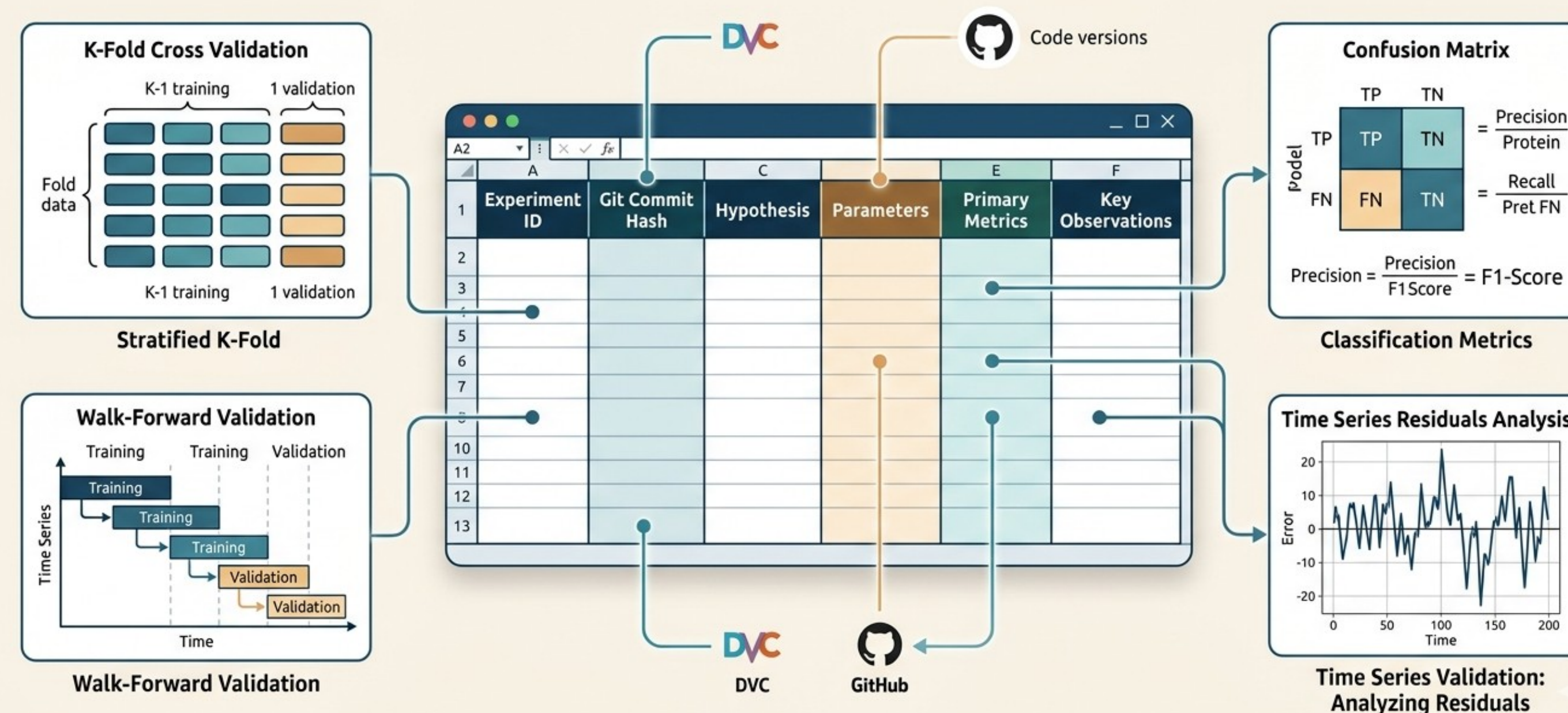
- Precision: "When I predict a 'Fraud', how accurate am I?"
- Recall: "How many of the actual 'Frauds' did I catch?"
- F1-Score: The harmonic mean

The "Visual Evidence" Link:

- Include a link or reference to the Confusion Matrix for every experiment.
- This may mean creating the matrix using code and saving it, or getting a screenshot.

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY

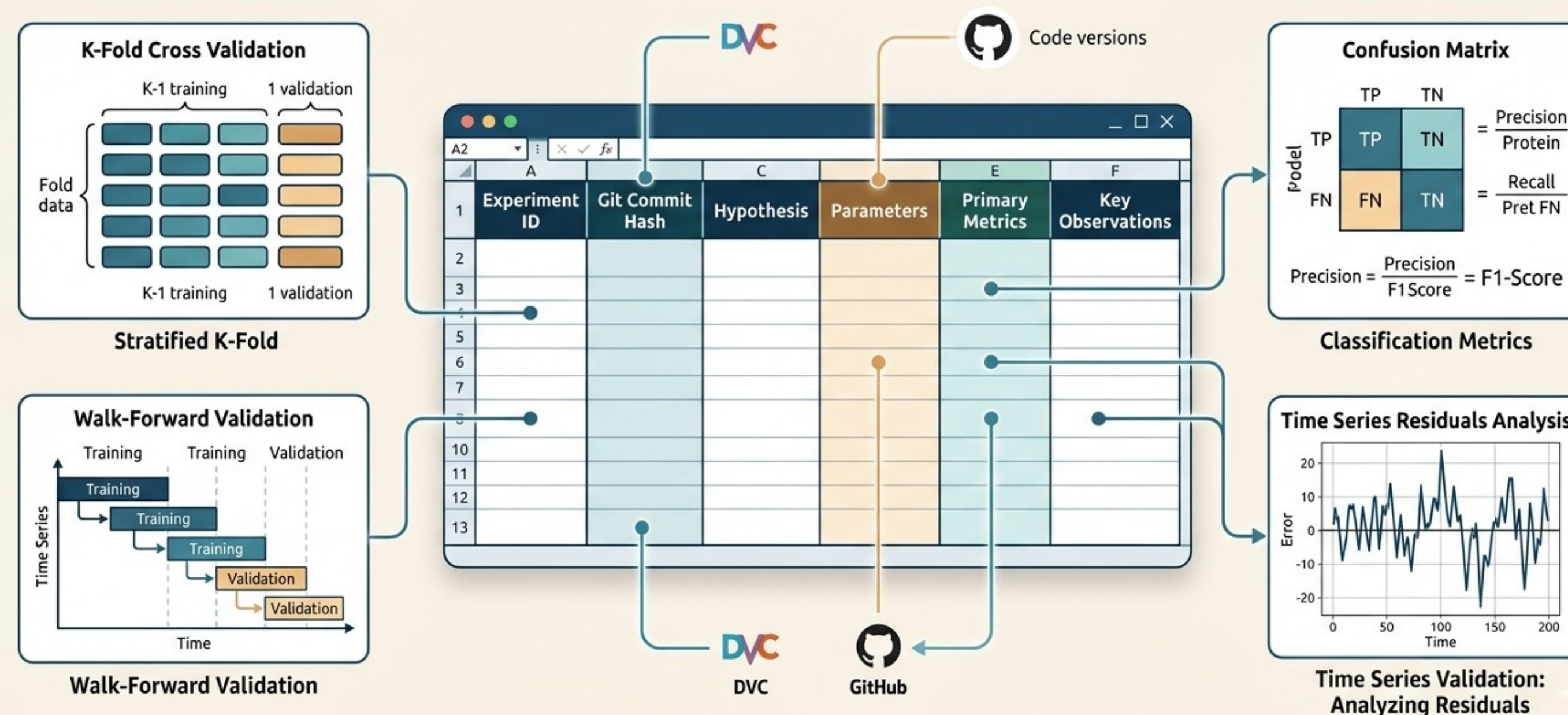


The Anatomy of an Experiment Ledger

Ledger Anatomy for Time Series Estimation: Tracking time series is predicting the future, so we focus on "Direction" and "Persistence."

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY



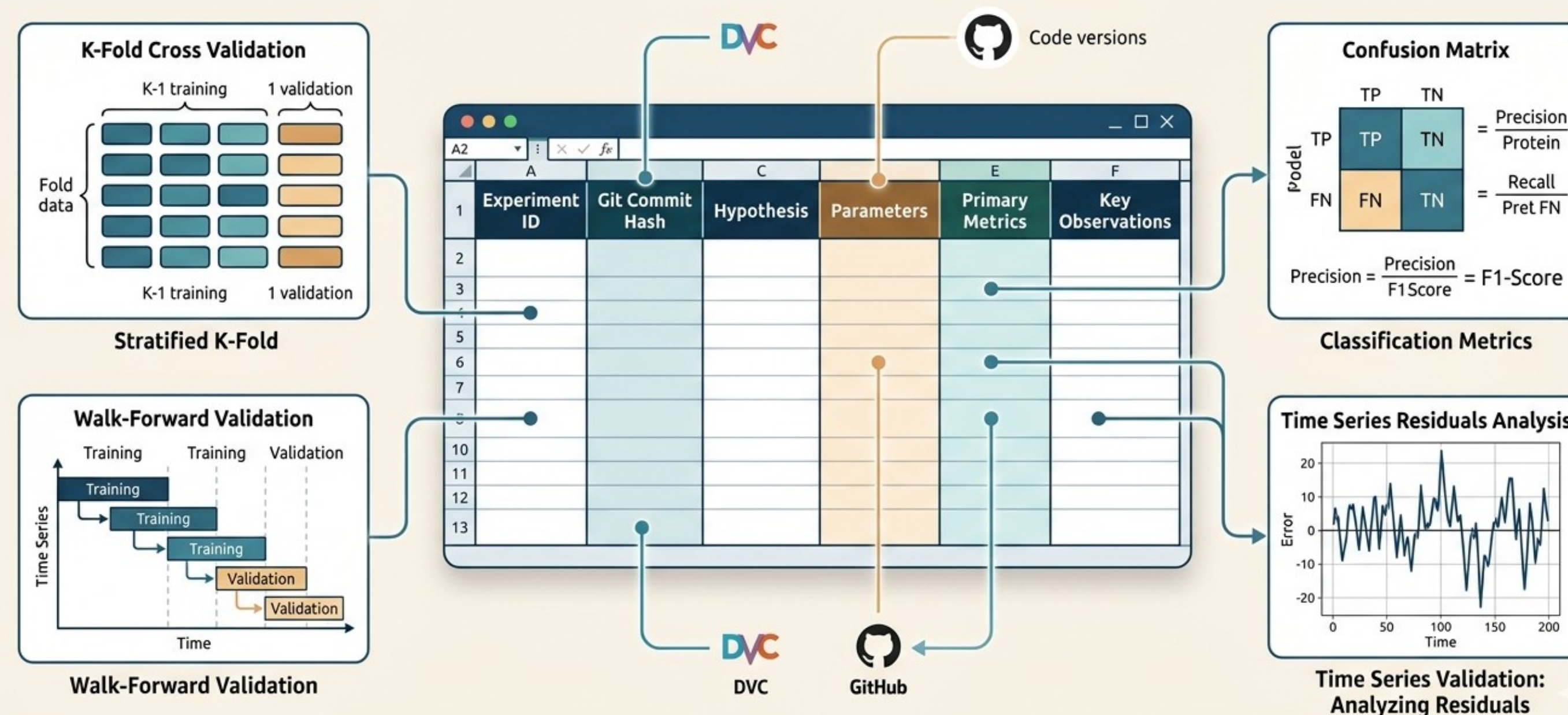
The Anatomy of an Experiment Ledger

Key Inputs to Record:

- Seasonality Period: (e.g., Weekly=7, Monthly=30).
- Lag Features: Did you use $t-1, t-2$ as inputs?
- Parameters: α, β, γ (Level, Trend, Seasonality).

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY



The Anatomy of an Experiment Ledger

Key Outputs (The Ledger Columns):

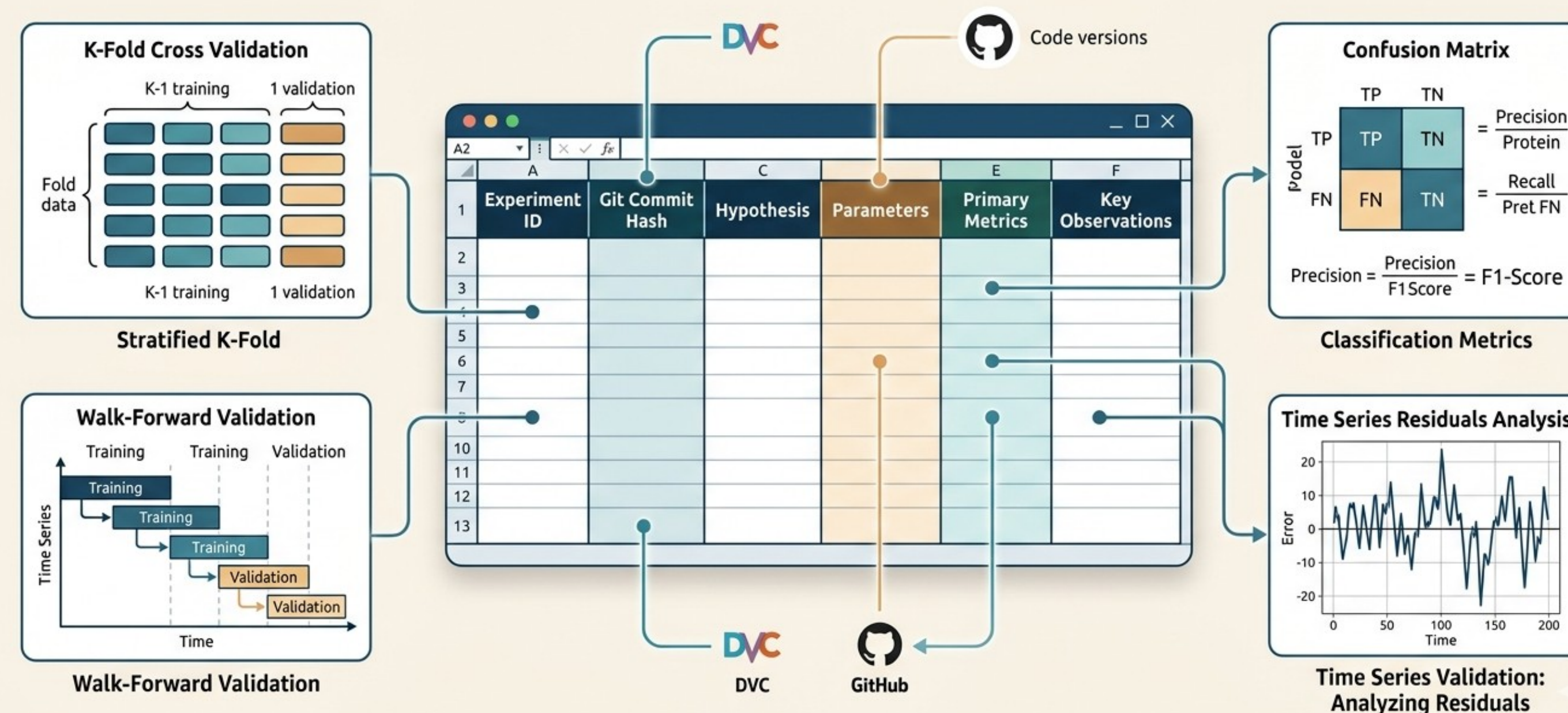
- MAPE (Mean Absolute % Error): Better than MSE for business.
- Directional Accuracy: Did the model predict the correct "Up/Down" move?
- Benchmark Test: Did it beat the "Naive Forecast"? (Today = Yesterday).

The "Visual Evidence" Link:

- Link to the Residuals Plot (Check if errors are random).

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY

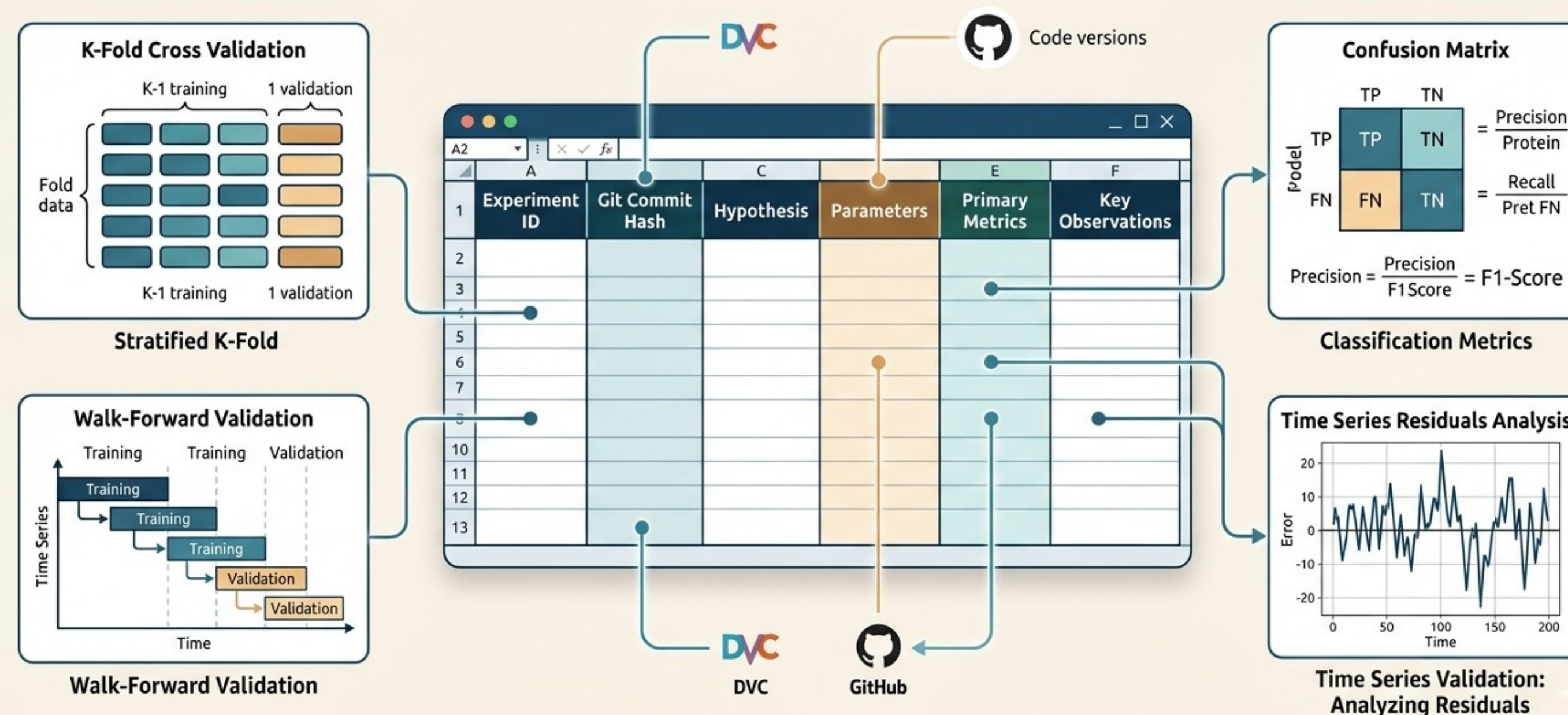


The Anatomy of an Experiment Ledger

Ledger Anatomy for Clustering: Tracking clusters is finding hidden patterns. So we focus on "Compactness" and "Separation."

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY



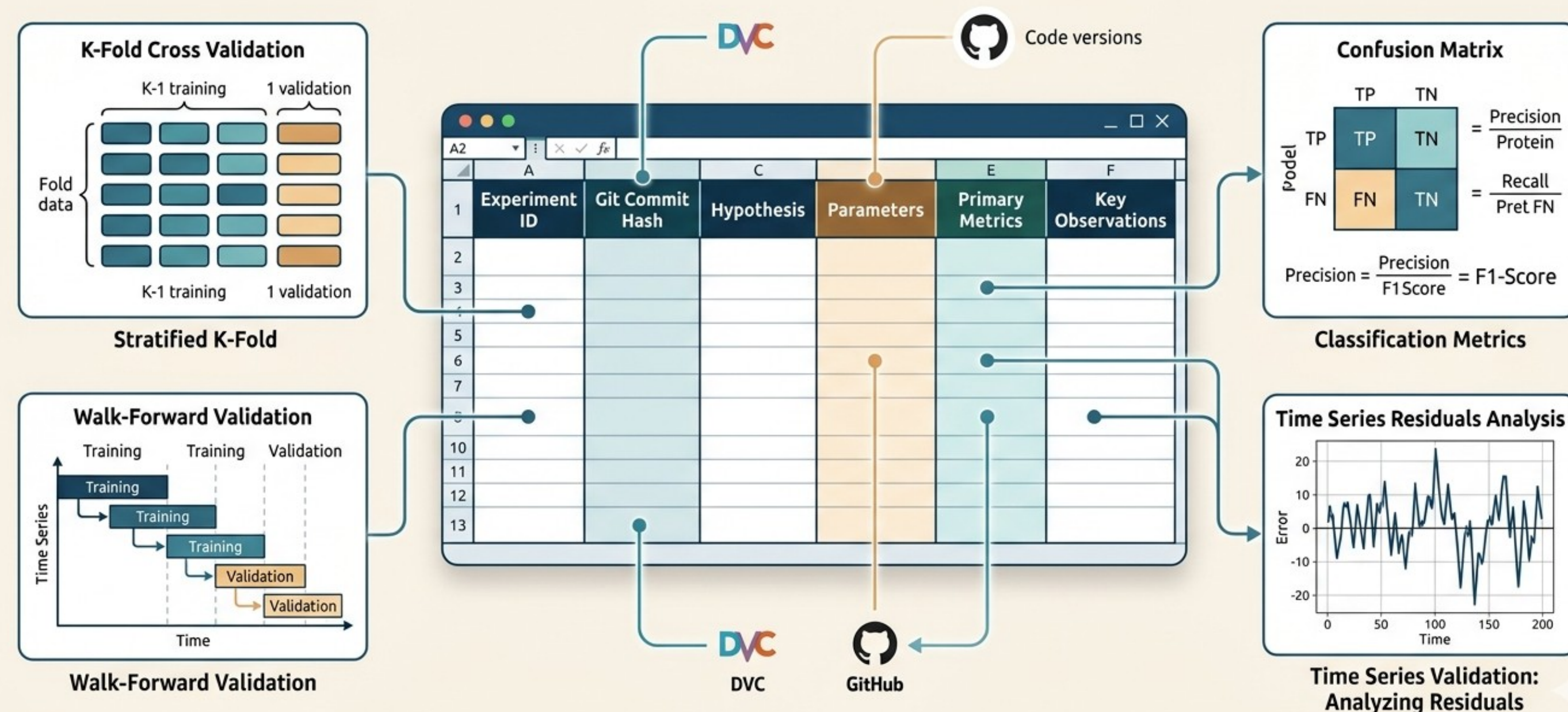
The Anatomy of an Experiment Ledger

Key Inputs to Record:

- Number of Clusters (K): How did you choose it?
- Distance Metric: Euclidean, Manhattan, or Cosine?
- Scaling Method: Min-Max vs. Standard Scaler (Crucial for distance).

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY



The Anatomy of an Experiment Ledger

Key Outputs (The Ledger Columns):

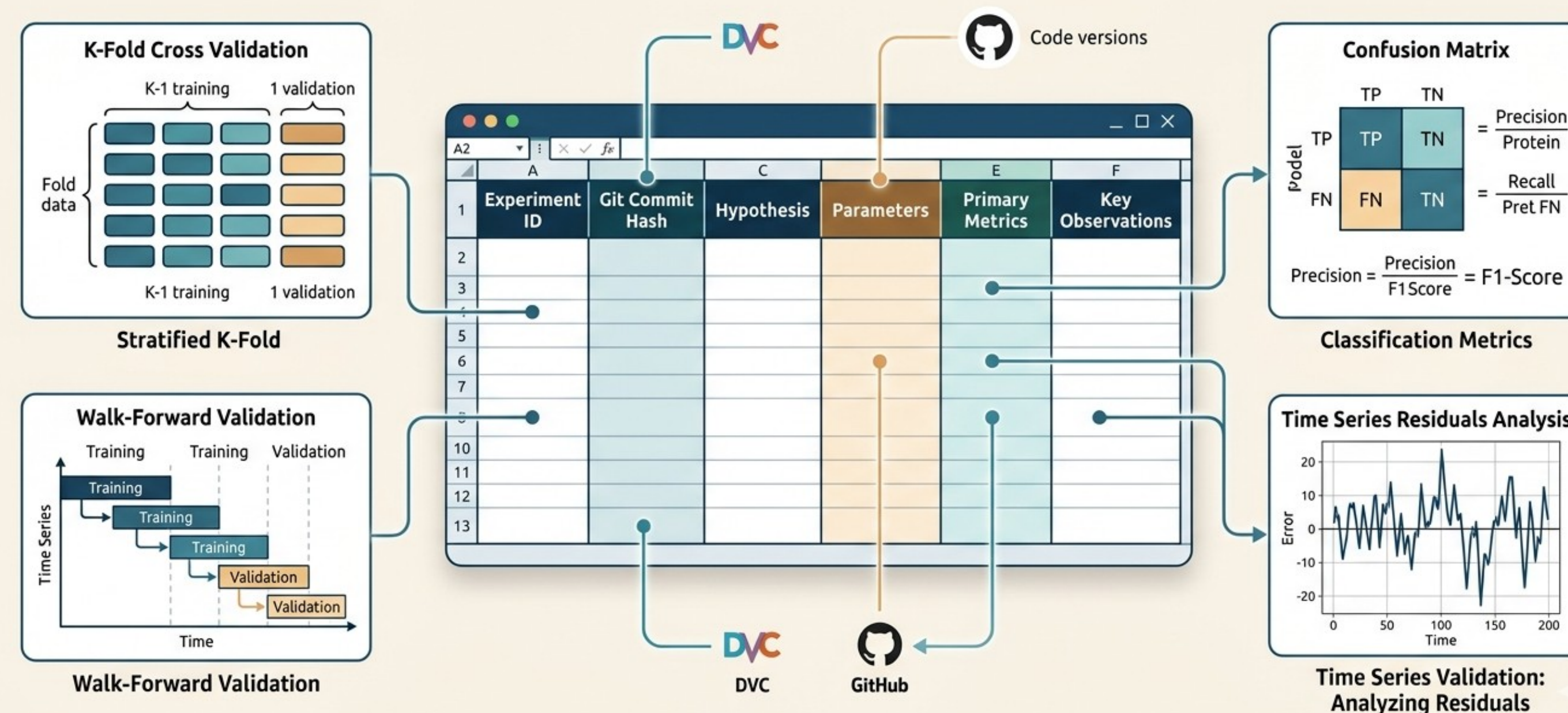
- Inertia (Within-Cluster Sum of Squares): How "tight" are the groups?
- Silhouette Score: How well-separated are the clusters?

The "Decision" Column:

- Record your interpretation: "Group 2 represents high-income, low-frequency buyers."

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY



The Anatomy of an Experiment Ledger

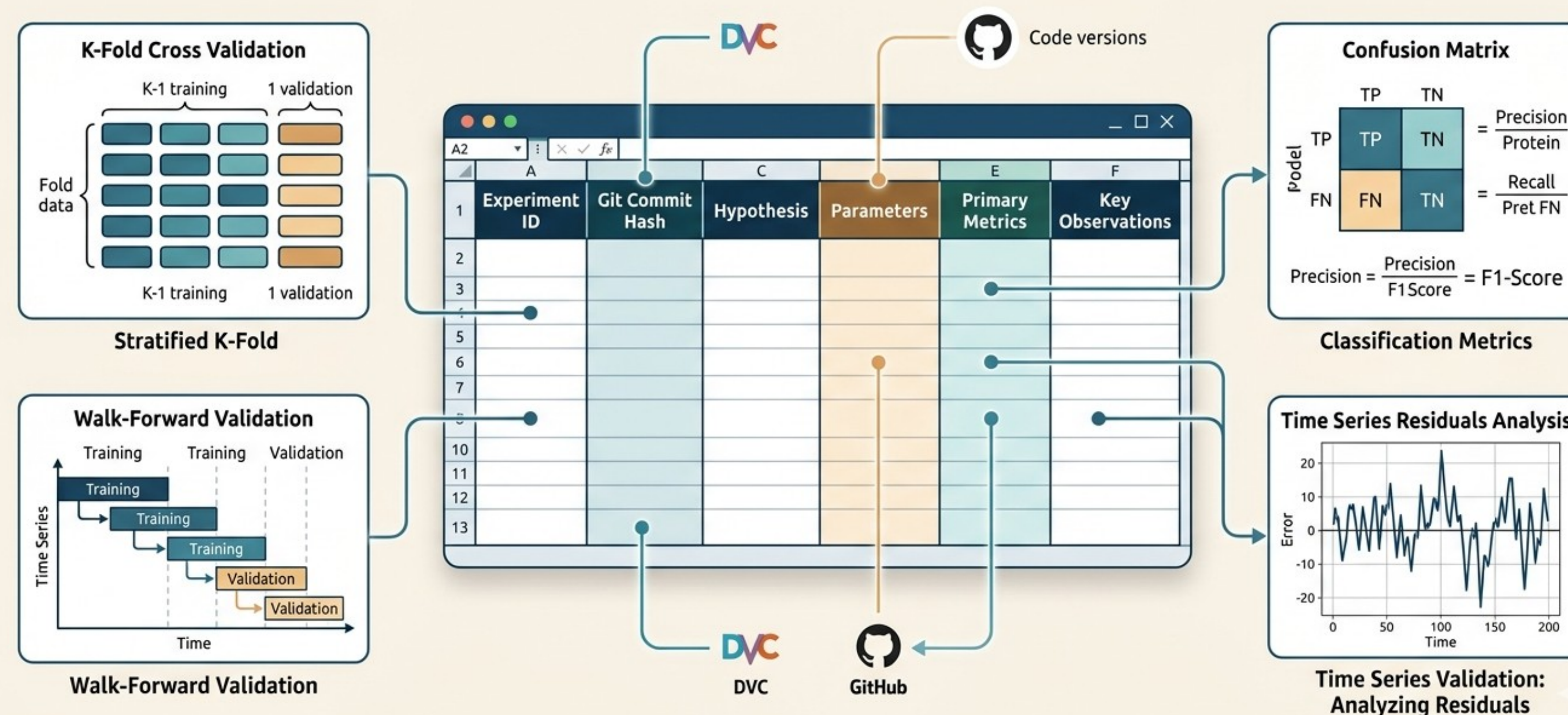
Putting it All Together: Which model version wins?

You have 10 rows in your Excel. Filter by goal.

- Classifier: "Show me the experiment with the highest Recall."
- Time Series: "Show me the experiment with the lowest MAPE."
- Clustering: "Show me the K value with the best Silhouette score."

THE ANATOMY OF AN EXPERIMENT LEDGER

A DISCIPLINED FRAMEWORK FOR INTEGRITY AND DISCOVERY

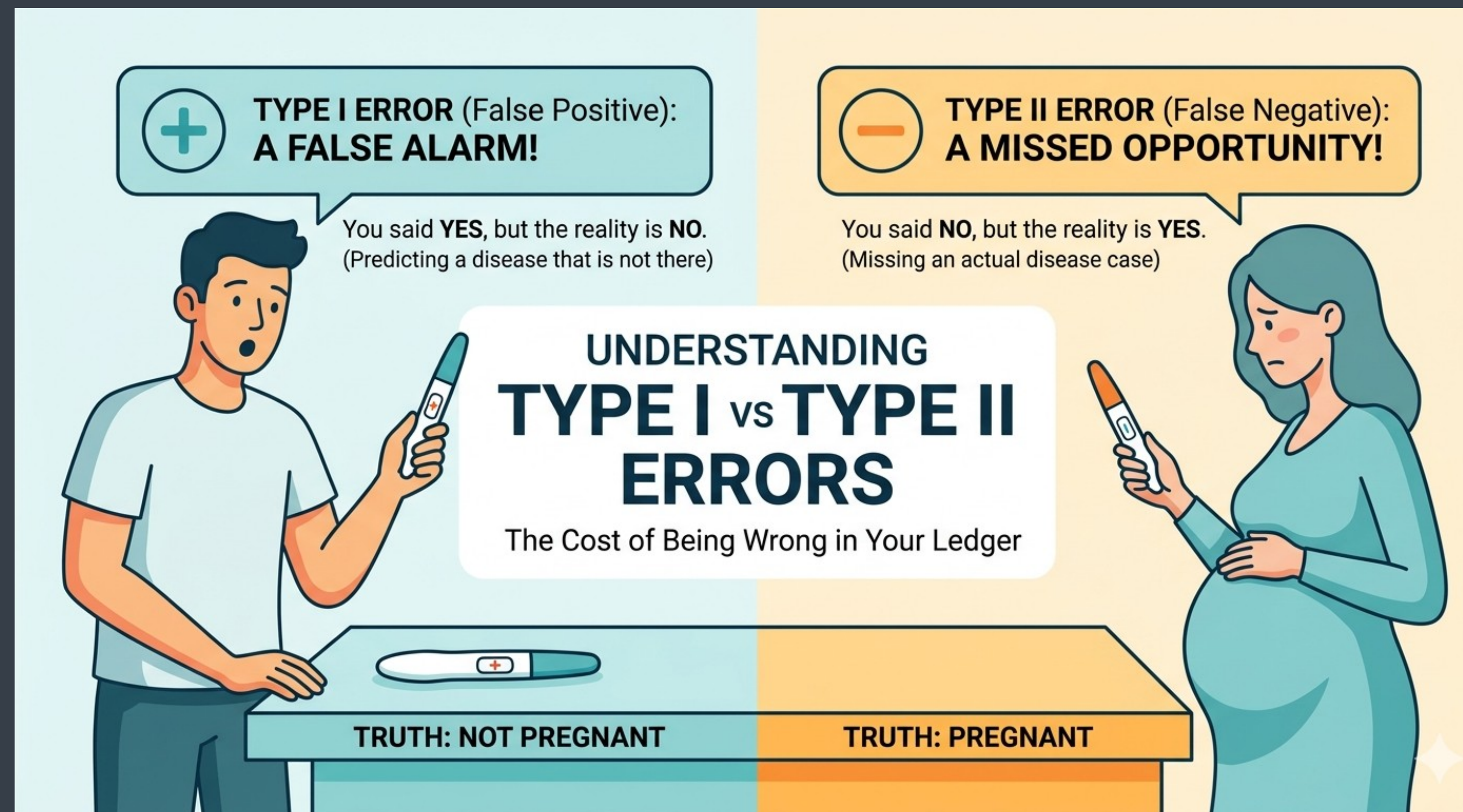


Classification: Beyond the Accuracy Trap

99% accuracy can be a failure because of imbalanced data.

Imagine a medical test for a rare disease that only 1% of people have.

- The "Lazy" Model: If your model simply predicts "Healthy" for everyone, it is 99% accurate.
- This model is useless because it missed 100% of the sick people.
- In the real world, "correctness" is not enough. You must look at where you fail.

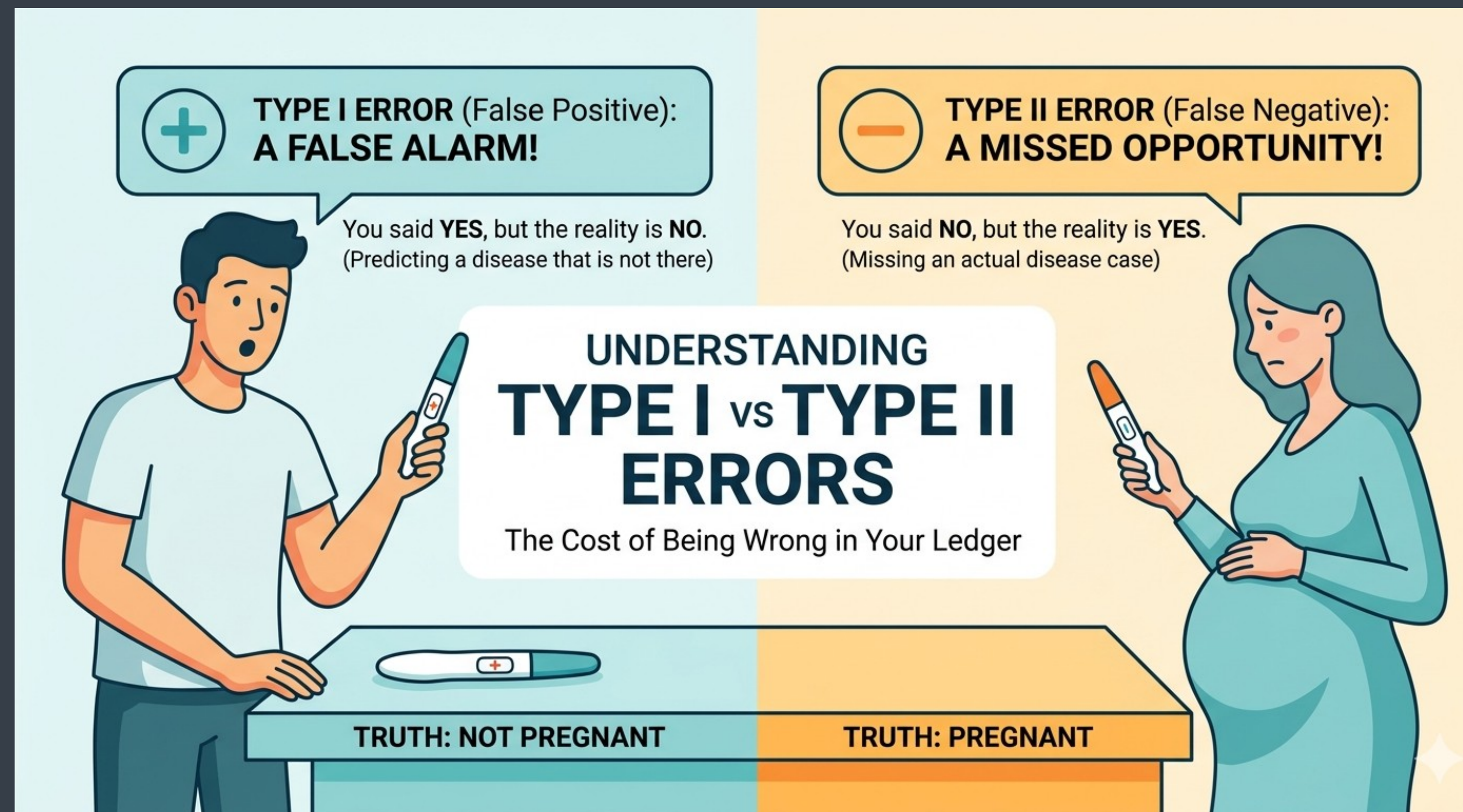


Classification: Beyond the Accuracy Trap

The Error Map (Confusion Matrix) is used to map your errors (Type I and Type II).

False Positive (Type I Error): A "False Alarm." You said "Yes," but the truth was "No."

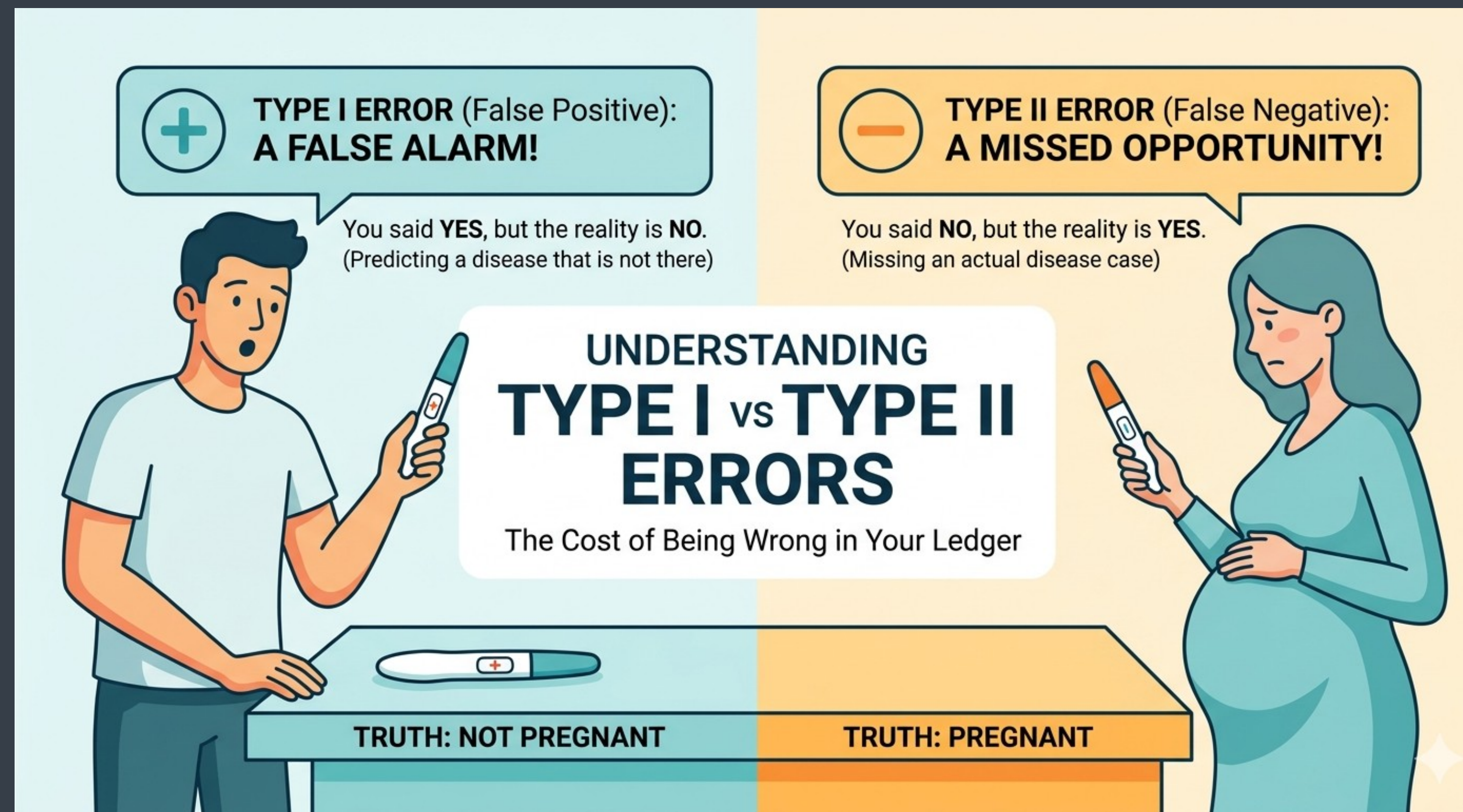
False Negative (Type II Error): A "Missed Opportunity." You said "No," but the truth was "Yes."



Classification: Beyond the Accuracy Trap

In Your Ledger: You must record these numbers to see the "character" of your model.

Usually, if you try to reduce False Positives, your False Negatives will increase. You must choose your side.

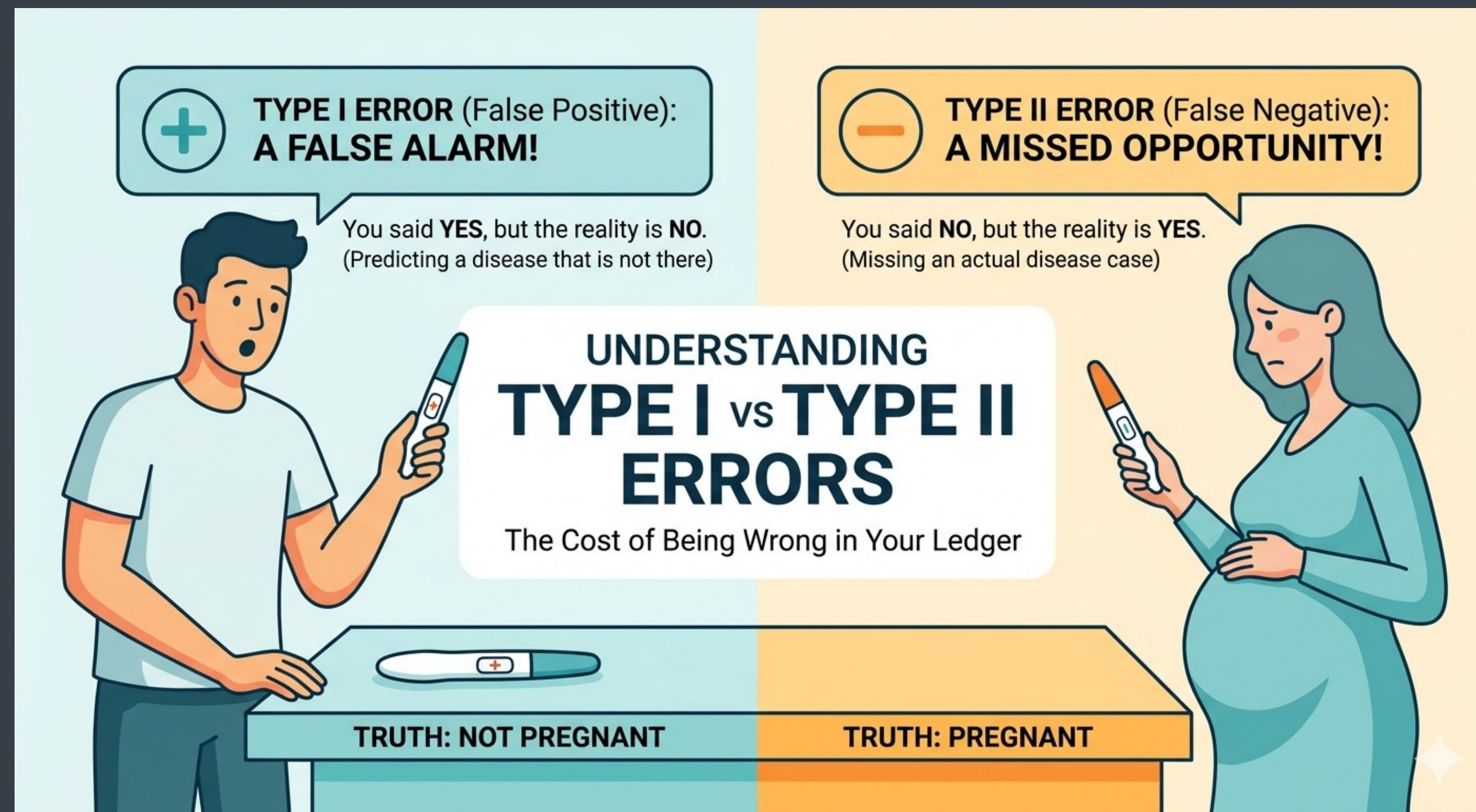


Classification: Beyond the Accuracy Trap

Precision (The Quality): "When I predict Positive, how often am I right?" Use this when False Positives are expensive (e.g., Sending an expensive marketing gift).

Recall (The Quantity): "Out of all actual Positives, how many did I find?" Use this when False Negatives are dangerous (e.g., Identifying a failing engine part).

F1-Score: The balance between the two. If you don't know which one to pick, start here.



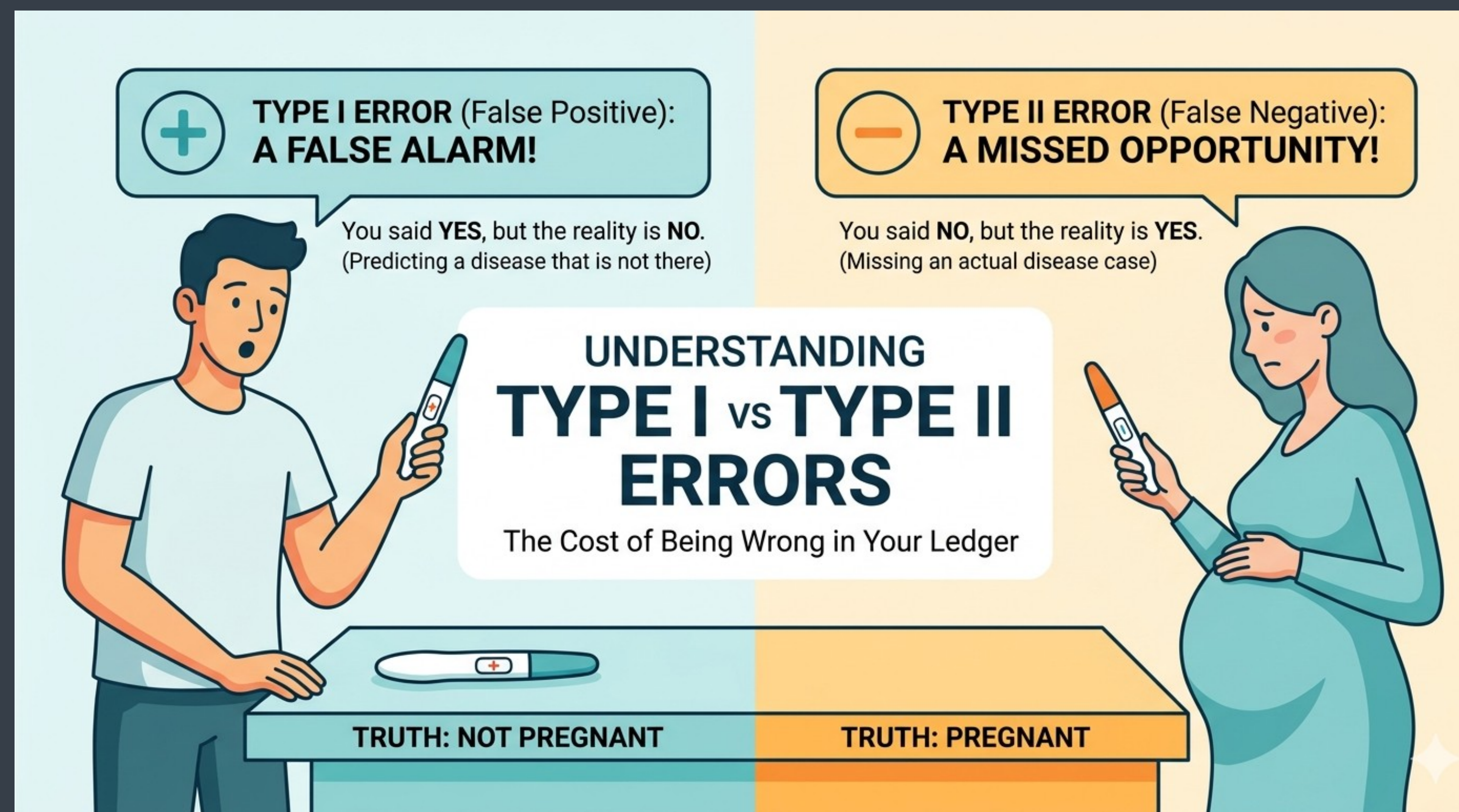
Classification: Beyond the Accuracy Trap

Recording "Truth" in your ledger also means you are justifying your model selection.

Don't just write "Accuracy: 0.85" in your Excel.

Mandatory Ledger Columns:

- Precision / Recall / F1
- Dominant Error Type: (FP or FN?)
- Business Impact: "I chose Model #4 because it has the highest Recall; we cannot afford to miss a single fraud case."

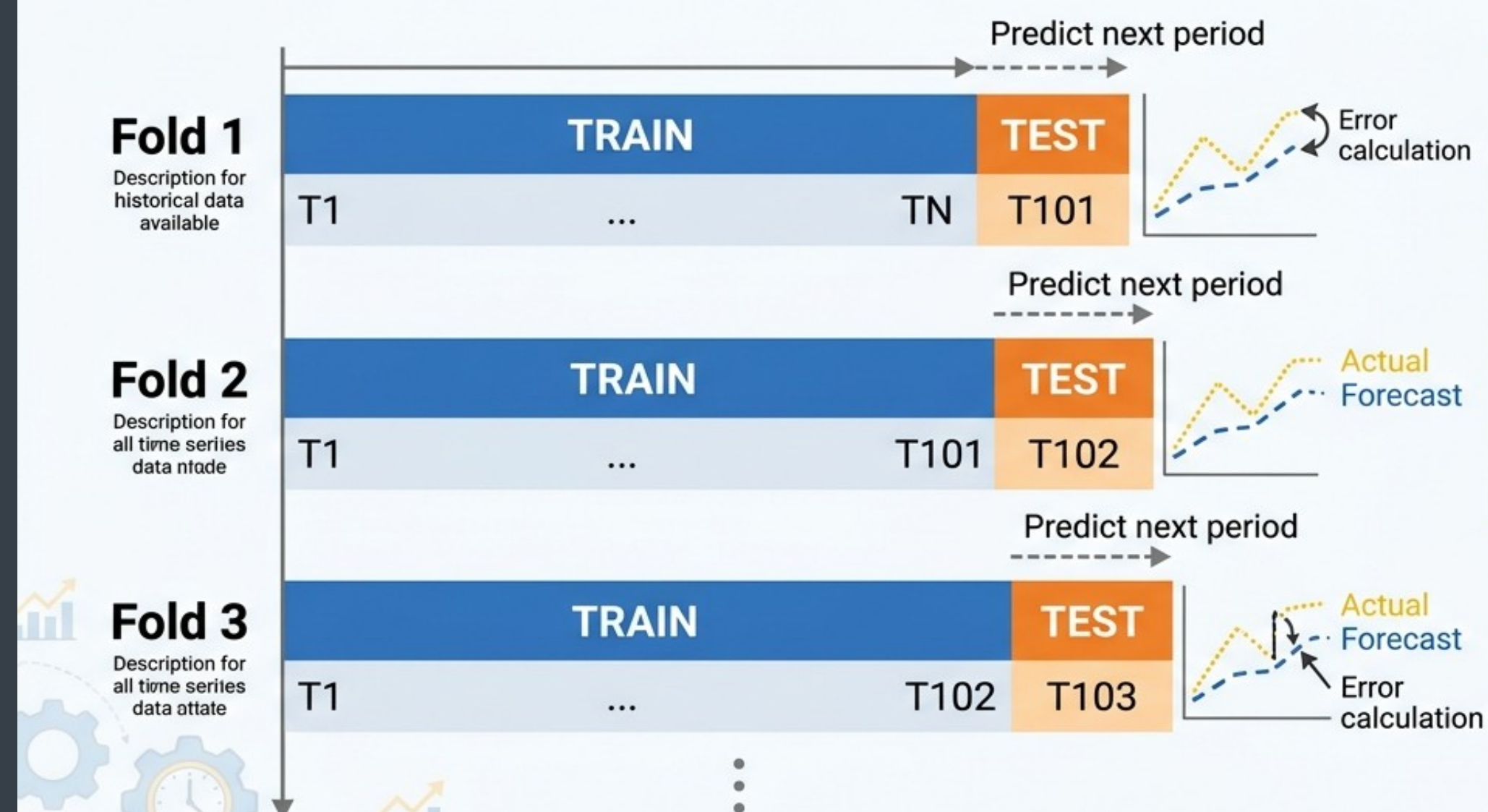


Time-Series: The "Future" Validation

The "Future" is not a random shuffle!
Hence the golden rule in time-series work:
"No Peeking!"

- The Mistake: In classification, we shuffle data. In Time-Series, shuffling is illegal.
- Why? You cannot use Wednesday's price to predict Monday's weather. Time has a direction.

Walk-Forward Validation: The Time-Series Method



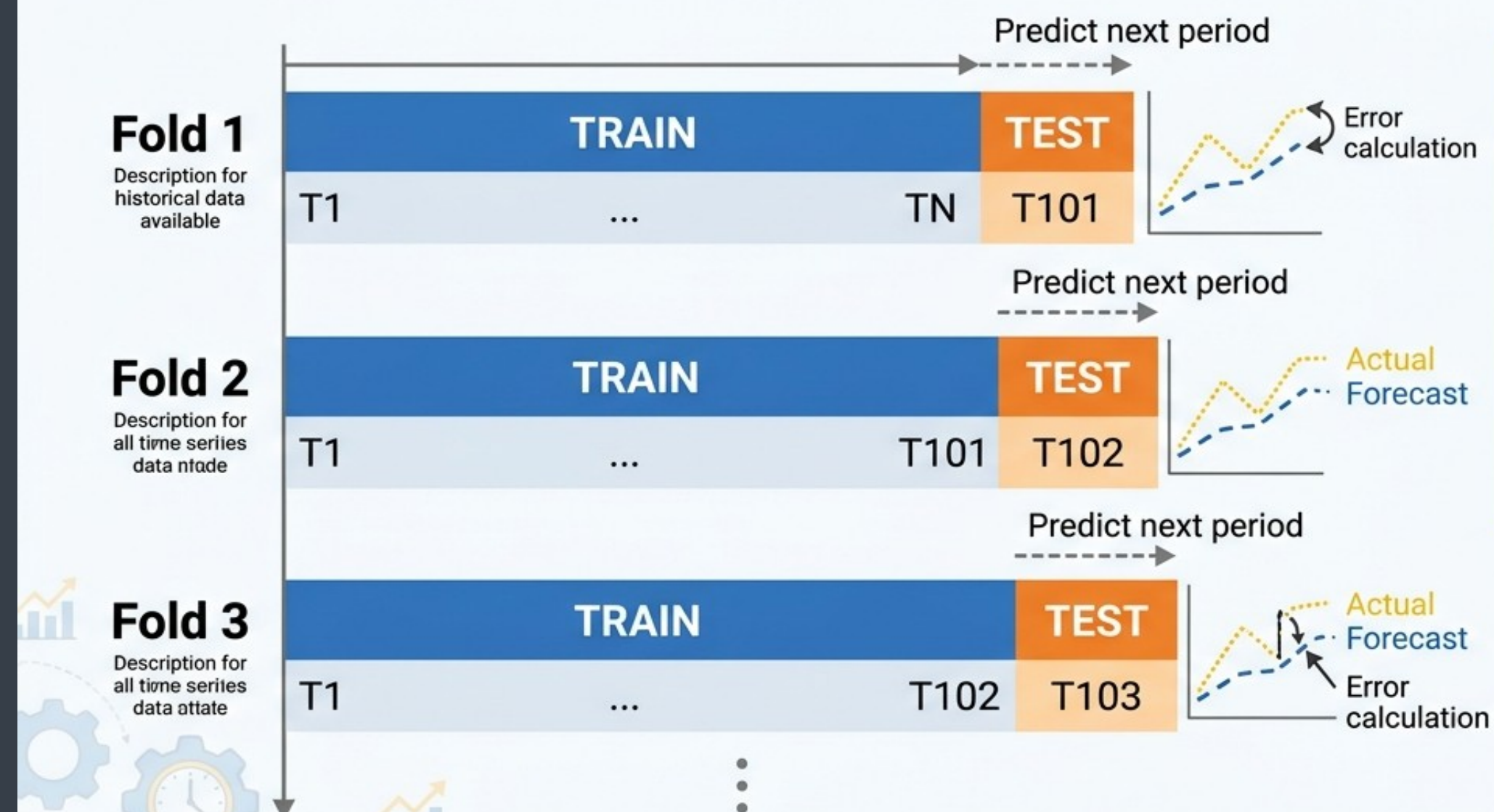
- Model trains on all historical data available.
- Model predicts the very next, unseen period.
- Error is recorded; data for this period is added to the training set for the next step.

Key Benefit: Evaluates performance in a true "future" scenario by rolling through time, preserving temporal order.

Time-Series: The "Future" Validation

- Data Leakage: If your model "sees" the future during training, your accuracy will be 100% in class (lab) but 0% in the real world.
- The Lesson: We must respect the arrow of time.

Walk-Forward Validation: The Time-Series Method



- Model trains on all historical data available.
- Model predicts the very next, unseen period.
- Error is recorded; data for this period is added to the training set for the next step.

Key Benefit: Evaluates performance in a true "future" scenario by rolling through time, preserving temporal order.

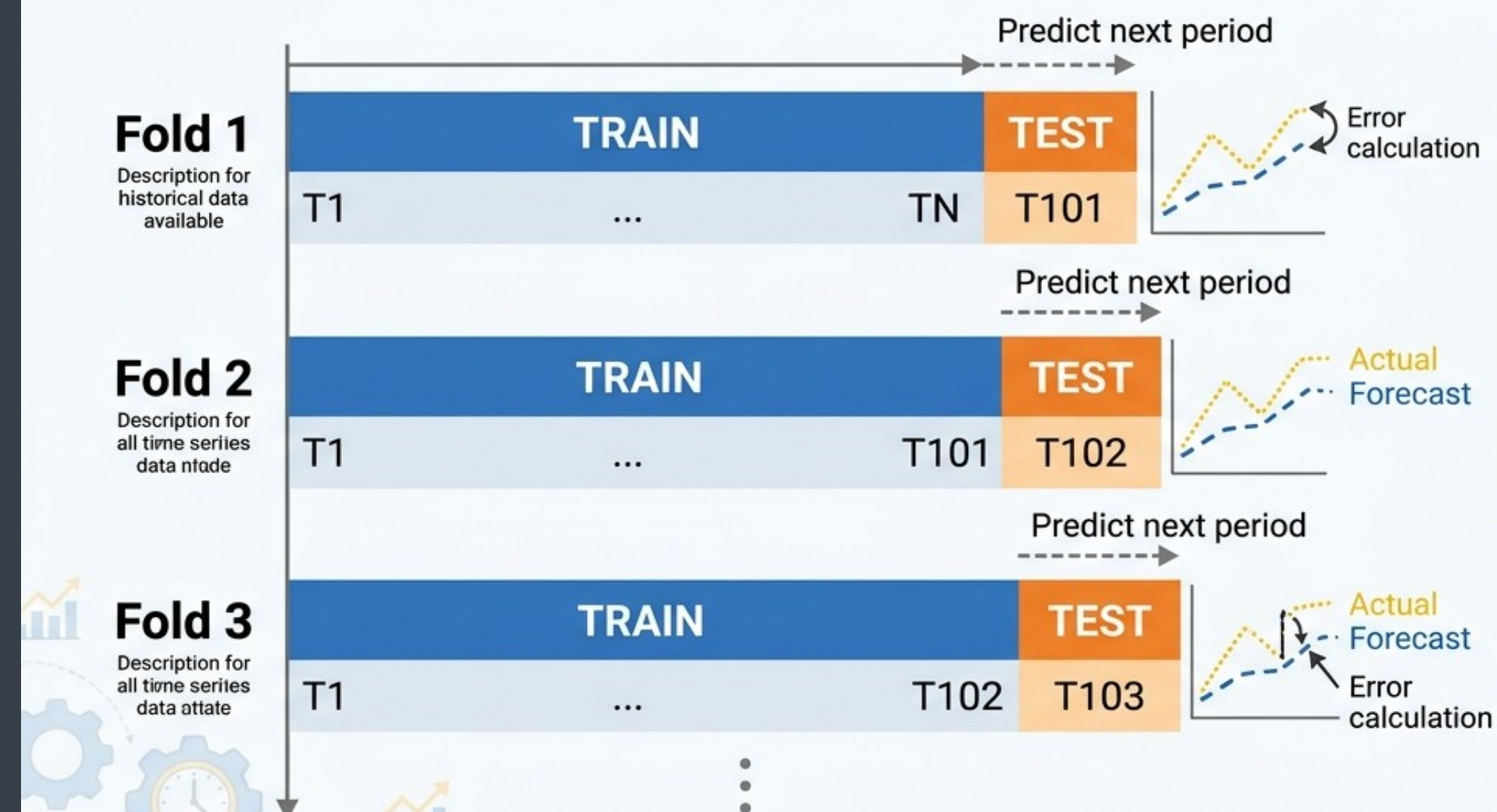
Time-Series: The "Future" Validation

“Walk-Forward Validation” is mimicking the world.

The Strategy: Instead of K-Fold, we use a "Sliding Window."

- Train on Month 1 → Test on Month 2
- Train on Month 1+2 → Test on Month 3
- Train on Month 1+2+3 → Test on Month 4

Walk-Forward Validation: The Time-Series Method



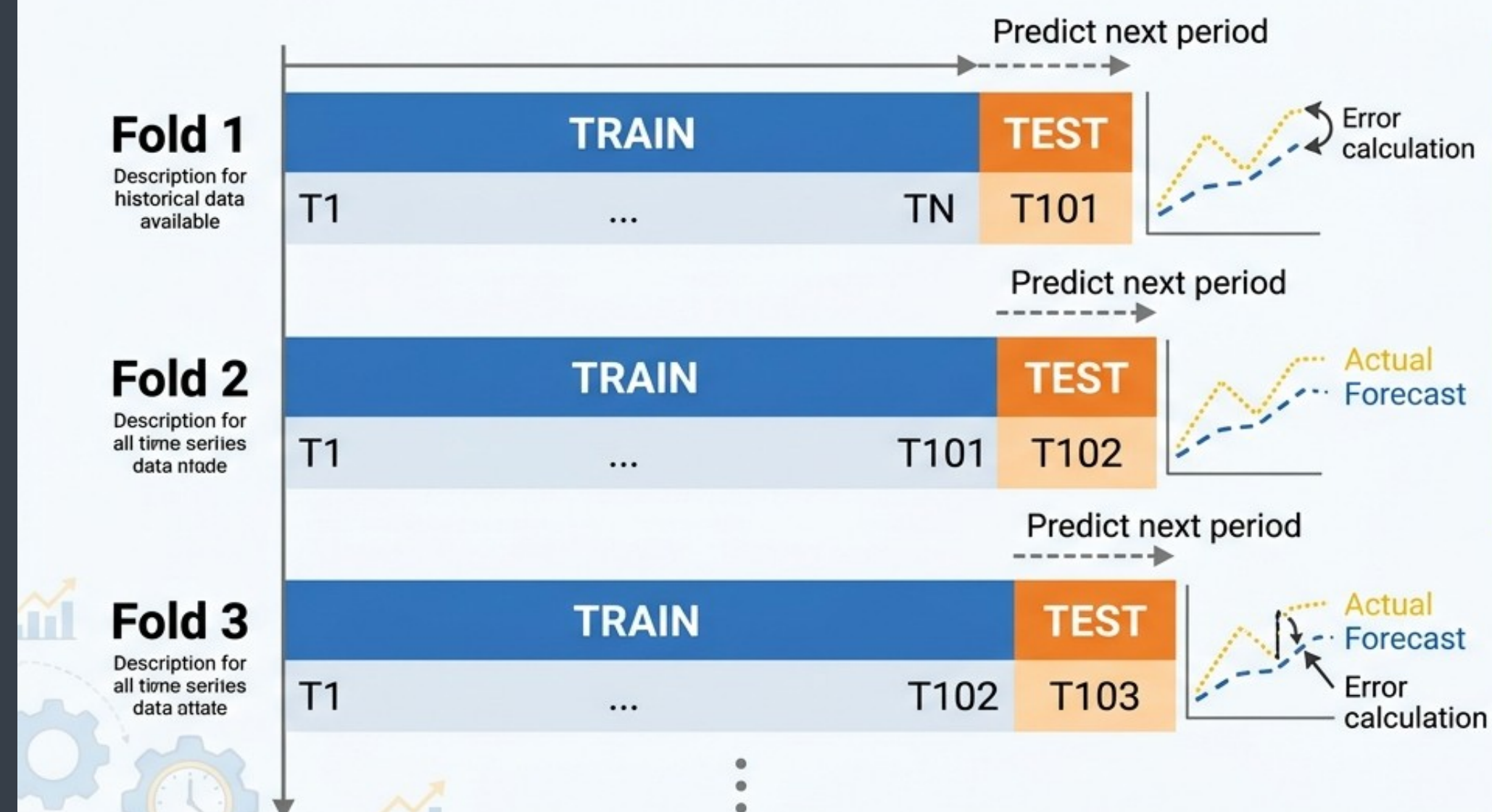
- Model trains on all historical data available.
- Model predicts the very next, unseen period.
- Error is recorded; data for this period is added to the training set for the next step.

Key Benefit: Evaluates performance in a true "future" scenario by rolling through time, preserving temporal order.

Time-Series: The "Future" Validation

- Each window is a separate experiment.
- In your Ledger: Each row represents a "window."
 - This shows if your model is consistent over time or just "lucky" for one month.

Walk-Forward Validation: The Time-Series Method



- Model trains on all historical data available.
- Model predicts the very next, unseen period.
- Error is recorded; data for this period is added to the training set for the next step.

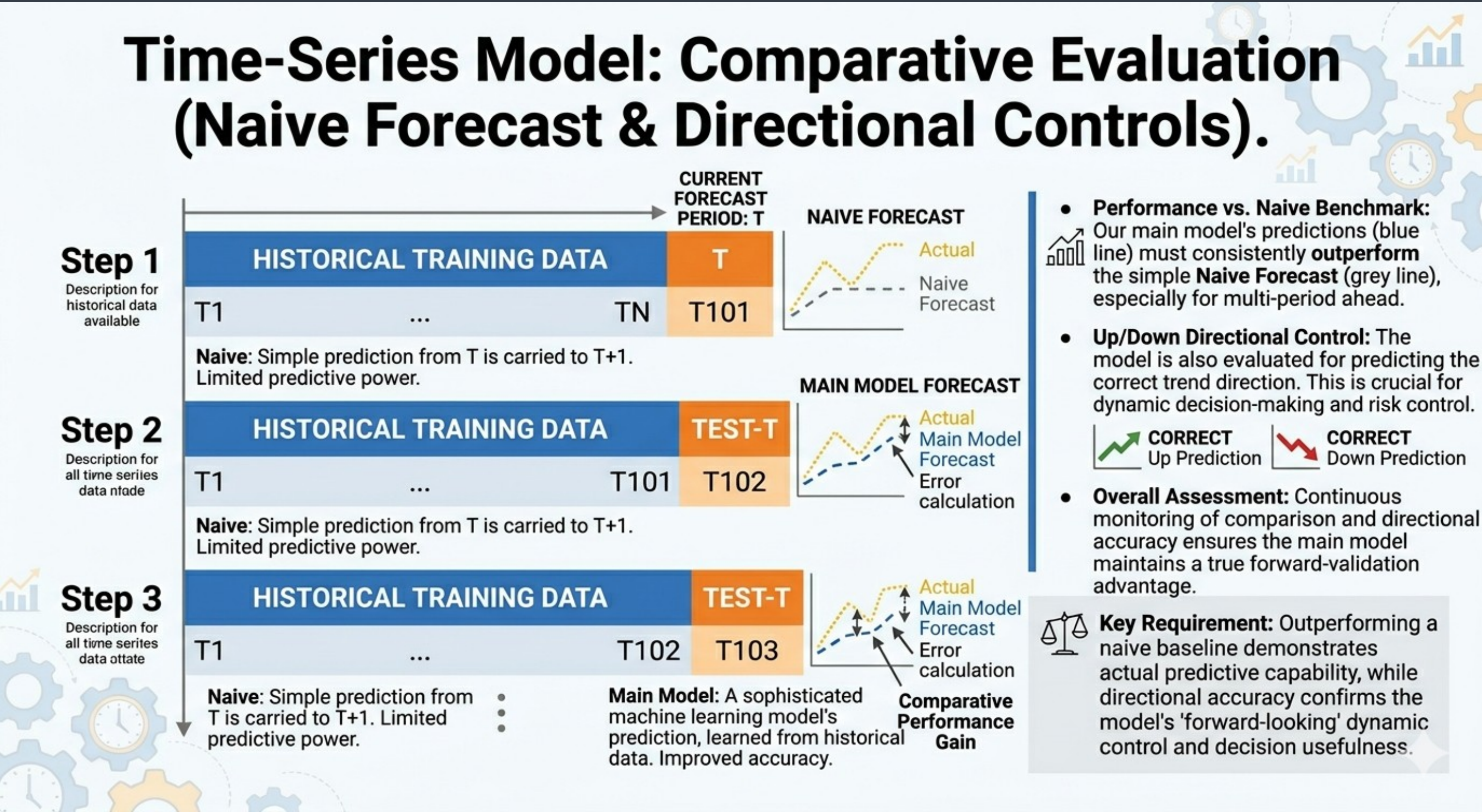
Key Benefit: Evaluates performance in a true "future" scenario by rolling through time, preserving temporal order.

Time-Series: The "Future" Validation

The "naive" reality check: Can you beat a "Random Guess"?

The Naive Forecast: A model that says "Tomorrow will be exactly like today."
($Value_{t+1} = Value_t$)

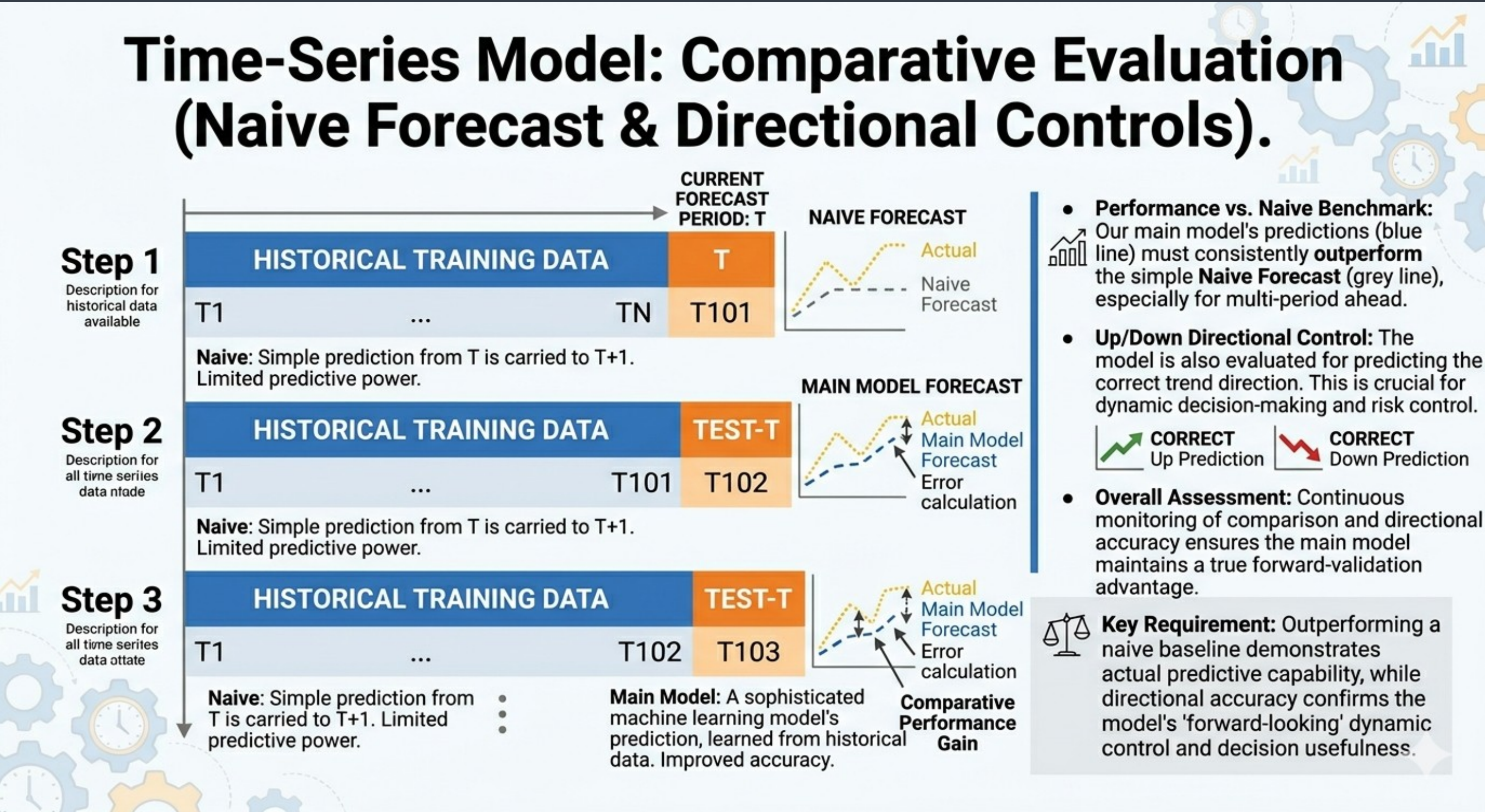
The Truth: If your complex model (Holt-Winters, LSTM, etc.) cannot beat this simple rule, your model is useless.



Time-Series: The "Future" Validation

Directional Accuracy: Don't just look at the error (MSE). Does your model correctly predict if the trend is UP or DOWN?

Business Impact: In logistics, missing the "direction" of a demand spike is more expensive than a small numerical error.



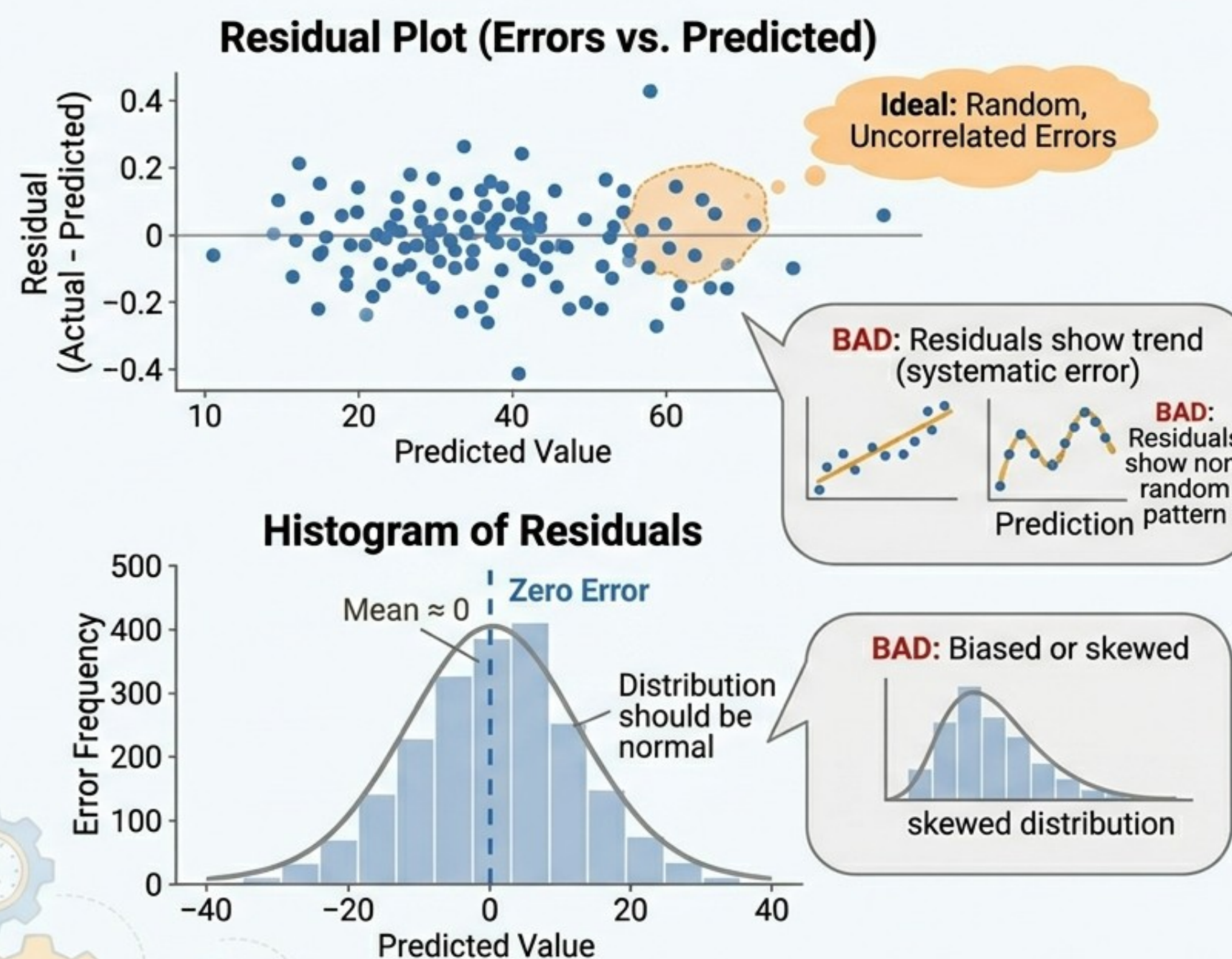
Time-Series: The "Future" Validation

Residual Analysis: What is left behind?

Residual = Actual Value – Predicted Value.
Residuals should be "White Noise"
(Random).

- If you see a pattern in your errors, your model is missing a feature (e.g., you forgot to add "Holidays").
- If errors are getting larger over time, your model is "drifting" (decay).

Residual Analysis: Core Diagnostics



- **Check Distribution:** Ideally, residuals should follow a standard Normal distribution (**mean ≈ 0**), confirming random variation is truly random.
- **Detect Patterns:** A well-functioning model must have non-correlated errors. Check the Residual Plot for **trends, clusters, or systematic biases** that require model refinement.
- **Iterative Improvement:** If systematic patterns or skew are found, re-examine features, model complexity, or data pre-processing steps.

Diagnostic Outcome: Successful residual diagnostics confirm the model's validity and ensure that all available, systematic information has been extracted, leaving only unexplainable noise.

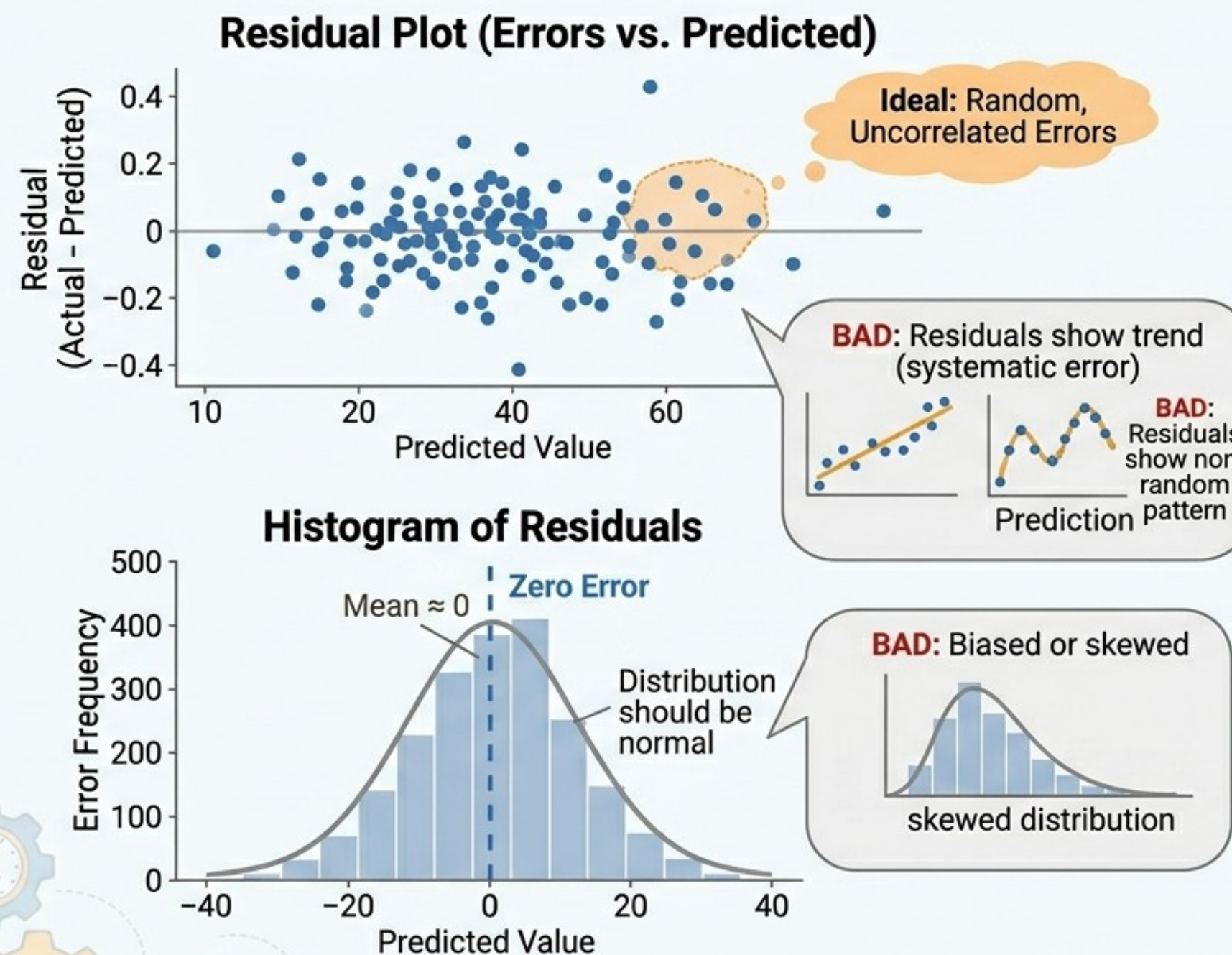
Time-Series: The "Future" Validation

The Ledger Rule: For every major experiment, look at your Residual Plot. If it's not random, don't trust the MSE.

If you suspect anything add columns to compare residual related metrics.

"Decision Integrity" is very important. Time-series estimation always has impact on profits, employment, health, etc.

Residual Analysis: Core Diagnostics



- **Check Distribution:** Ideally, residuals should follow a standard Normal distribution (**mean ≈ 0**), confirming random variation is truly random.
- **Detect Patterns:** A well-functioning model must have non-correlated errors. Check the Residual Plot for **trends, clusters, or systematic biases** that require model refinement.
- **Iterative Improvement:** If systematic patterns or skew are found, re-examine features, model complexity, or data pre-processing steps.

Diagnostic Outcome: Successful residual diagnostics confirm the model's validity and ensure that all available, systematic information has been extracted, leaving only unexplainable noise.

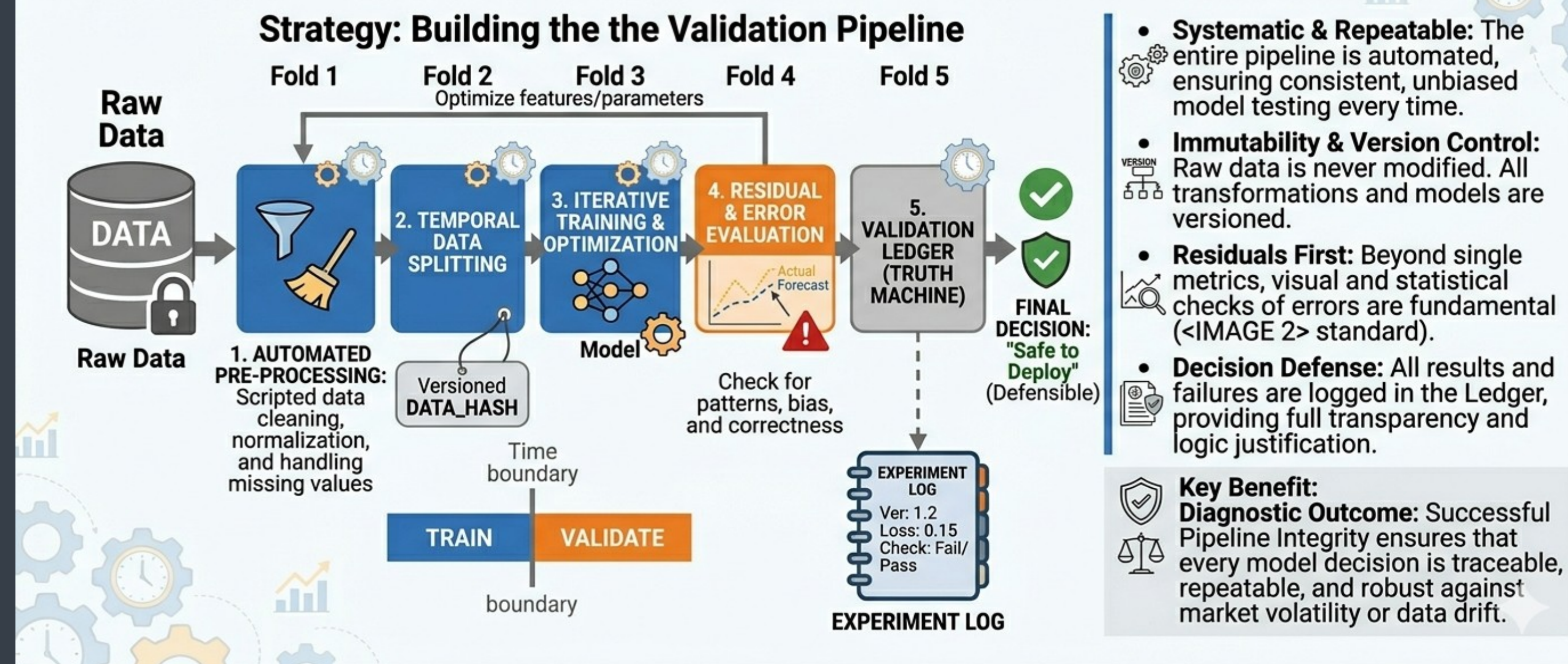
Strategy: Building the Validation Pipeline

Main Idea: Validation is not a one-time event; it is a repeatable, automated process.

- Automation: Every step, from data cleaning to scoring, must be scripted. No manual "copy-pasting" results.

- Immutability: Raw data stays original. Every transformation creates a new, versioned step.

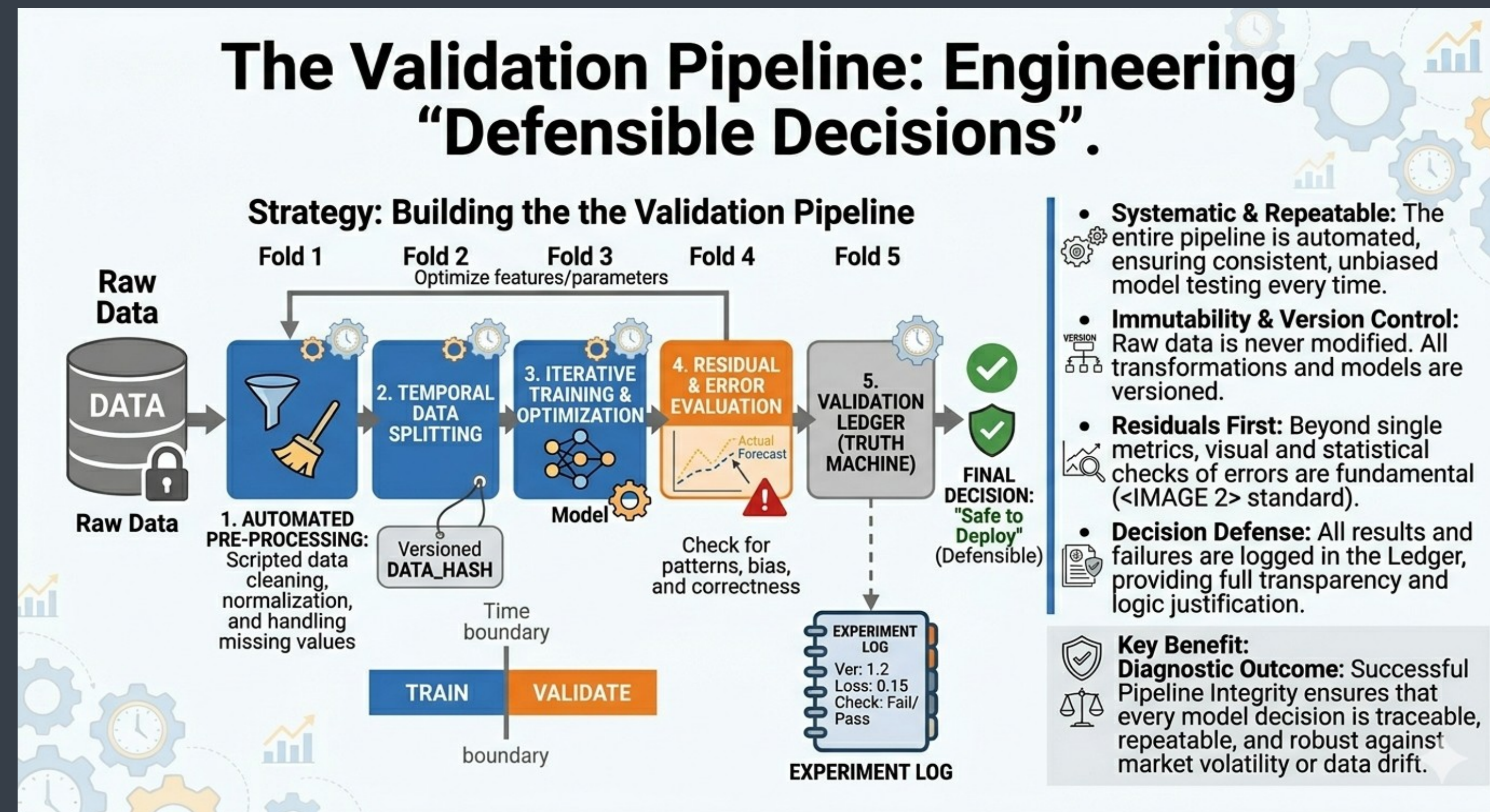
The Validation Pipeline: Engineering "Defensible Decisions".



Strategy: Building the Validation Pipeline

Main Idea: Validation is not a one-time event; it is a repeatable, automated process. (cnt'd)

- Systematic Flow: A pipeline ensures that your model is tested exactly the same way every time you change a parameter.
- Key takeaway: If you cannot repeat the process with one click, it is not a pipeline.



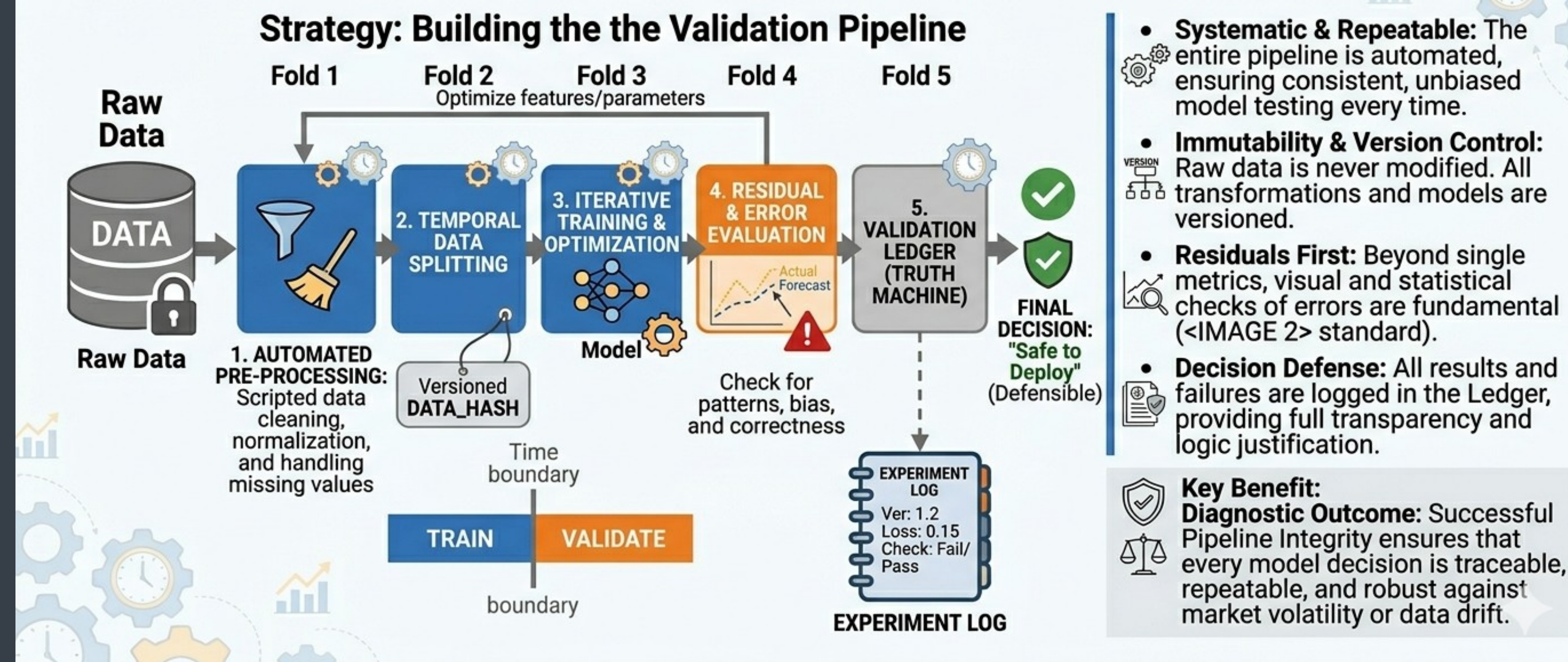
Strategy: Building the Validation Pipeline

The "Decision Defense" Framework: Your goal is not just a "good score," but a "defensible decision."

Statistical vs. Business Reality: A 5% error might be "statistically" small, but what is the cost of that error in the real world?

Stress Testing: How does the model perform if the data becomes "noisy" or if a sensor fails?

The Validation Pipeline: Engineering "Defensible Decisions".

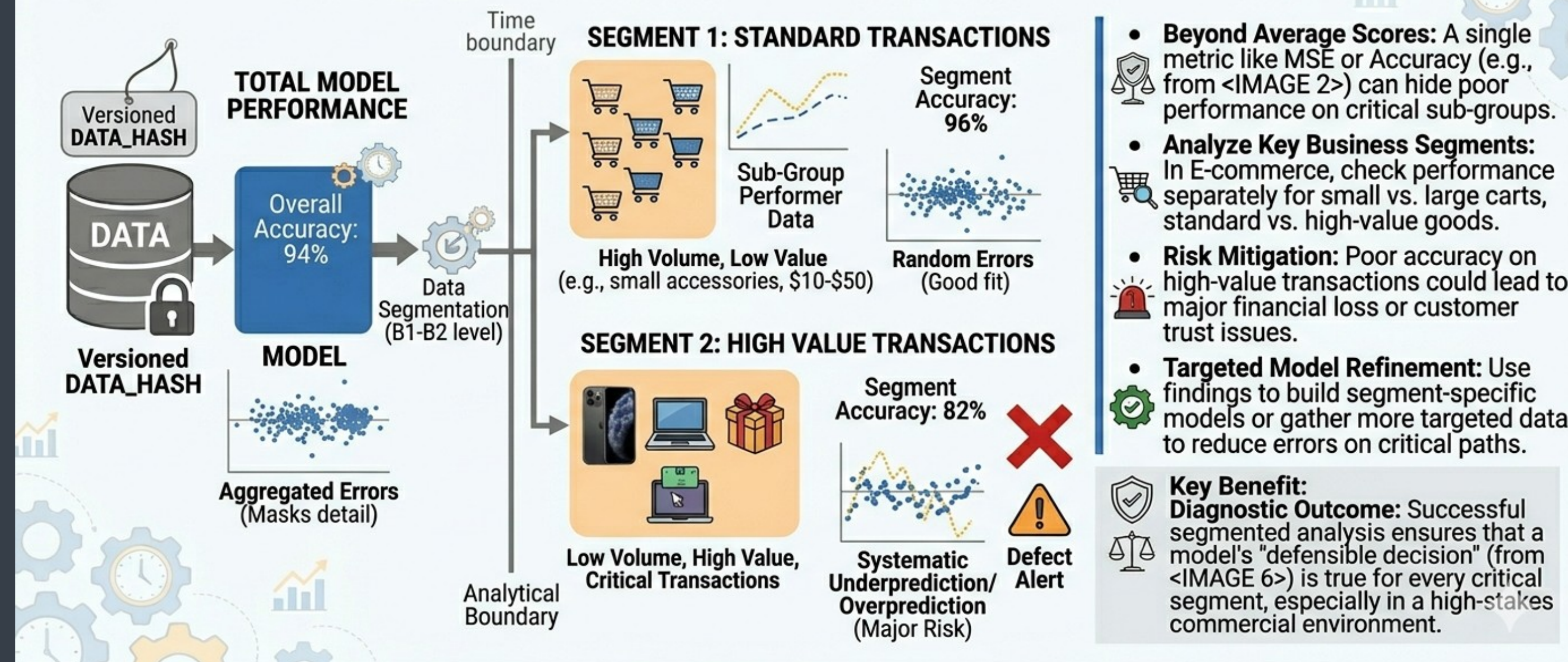


Strategy: Building the Validation Pipeline

Segmented Accuracy: Does the model work well for all groups, or does it fail on specific important cases (e.g., high-value transactions)?

The Auditor's Question: Could you explain to a manager why this model is safe to deploy?

Segmented Accuracy: Uncovering Sub-Group Performance in E-commerce



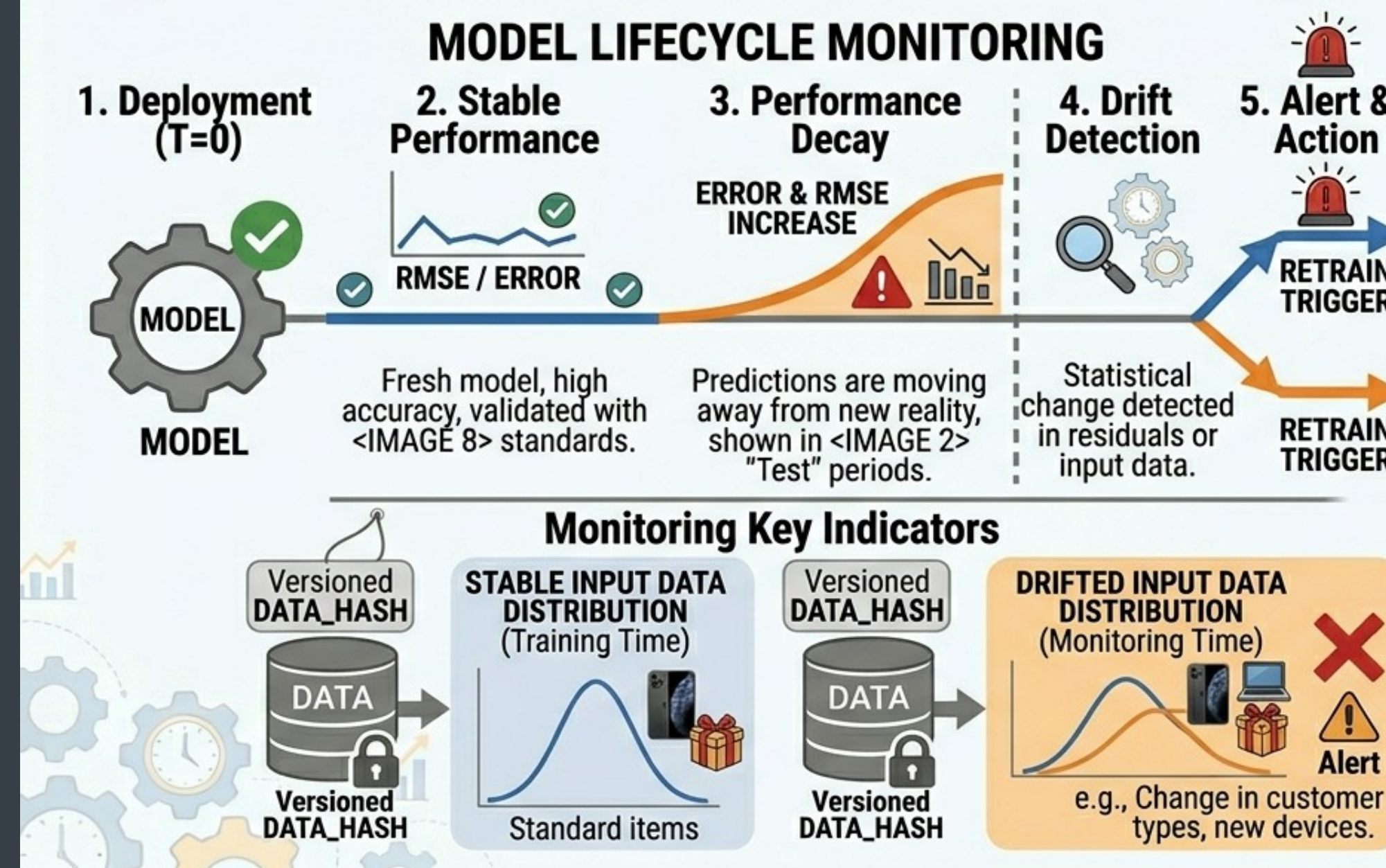
Strategy: Building the Validation Pipeline

Detecting "Model Decay" (The Drift):

Models are like milk; they expire as the world changes.

- Concept Drift: When the relationship between input and output changes (e.g., shopping habits after a global event).
- Data Drift: When the statistical properties of your input change (e.g., your users' average age increases).

Detecting Model Decay (Model Drift)



- **Beyond Average Scores:** A single metric like MSE or Accuracy (from <IMAGE 2>) can hide poor performance on critical sub-groups.
- **Concept vs. Data Drift:** Concept drift (e.g., changes in post-event behavior from <IMAGE 2>) is an intrinsic model loss. Data drift (e.g., changes in devices from <IMAGE 8>) is an input shift.
- **Risk Mitigation: Retraining Triggers.** Set limits (e.g., 20% RMSE increase) or monitor standard patterns.
- **Targeted Model Refinement:** Use findings to build segment-specific models or gather more targeted data from <IMAGE 8> paths.

Key Benefit:
Diagnostic Outcome: Successful model lifecycle monitoring ensures that every "defensible decision" (from <IMAGE 6>) is true through time, especially in a high-stakes commercial environment like <IMAGE 8>.

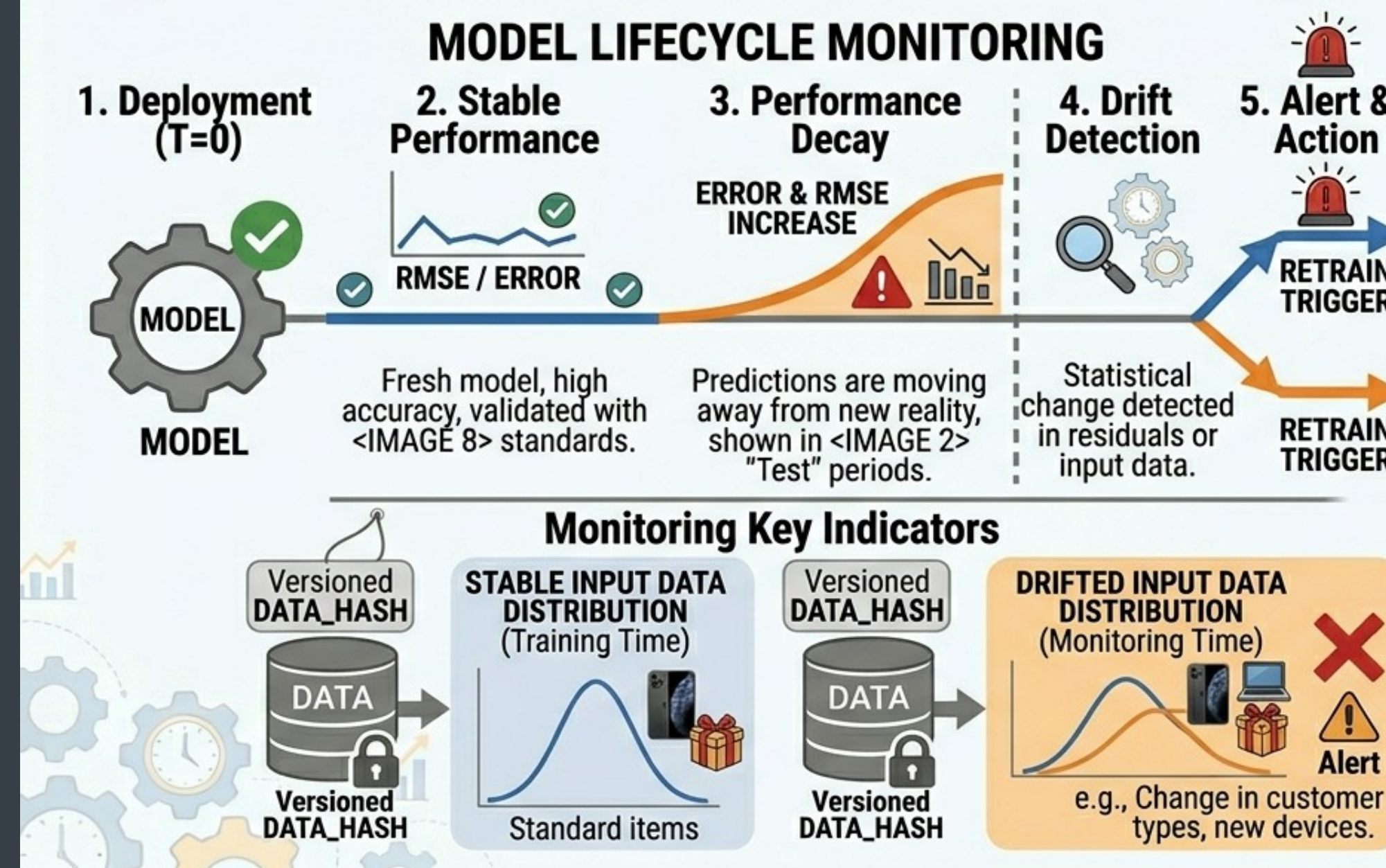
Strategy: Building the Validation Pipeline

Monitoring Residuals: If your residuals (errors) start showing a pattern over time, the model is losing its "knowledge."

Retraining Strategy: Decide when to retrain: every week? Or when the error passes a specific limit?

Use your ledger for these decisions!

Detecting Model Decay (Model Drift)



- **Beyond Average Scores:** A single metric like MSE or Accuracy (from <IMAGE 2>) can hide poor performance on critical sub-groups.
 - **Concept vs. Data Drift:** Concept drift (e.g., changes in post-event behavior from <IMAGE 2>) is an intrinsic model loss. Data drift (e.g., changes in devices from <IMAGE 8>) is an input shift.
 - **Risk Mitigation:** Retraining Triggers. Set limits (e.g., 20% RMSE increase) or monitor standard patterns.
 - **Targeted Model Refinement:** Use findings to build segment-specific models or gather more targeted data from <IMAGE 8> paths.
- Key Benefit:**
Diagnostic Outcome: Successful model lifecycle monitoring ensures that every "defensible decision" (from <IMAGE 6>) is true through time, especially in a high-stakes commercial environment like <IMAGE 8>.

Strategy: Building the Validation Pipeline

Using the "Lab Notebook" mentality within the pipeline is very useful.

Version Control: Save the code, the hyperparameters, and the "hash" of the dataset used.

Failure Documentation: Record every "bad" model. Knowing what didn't work is as important as knowing what did.



Strategy: Building the Validation Pipeline

Transparency: A ledger allows anyone to verify your results. It prevents "cherry-picking" only the best scores.

Integrity: In professional environments, the Ledger is your insurance against accusations of data manipulation.



Strategy: Building the Validation Pipeline

Prepare concrete rules for your projects and treat them like a checklist.

Example rules:

- Baseline First: Always compare your model against a Naive Forecast.
- No Leaks: Ensure no future data is visible to the model during training.
- Visual Check: Never trust a single number (MSE/MAE); always look at the Residual Plot.



Strategy: Building the Validation Pipeline

Prepare concrete rules for your projects and treat them like a checklist.

Example rules (cnt'd):

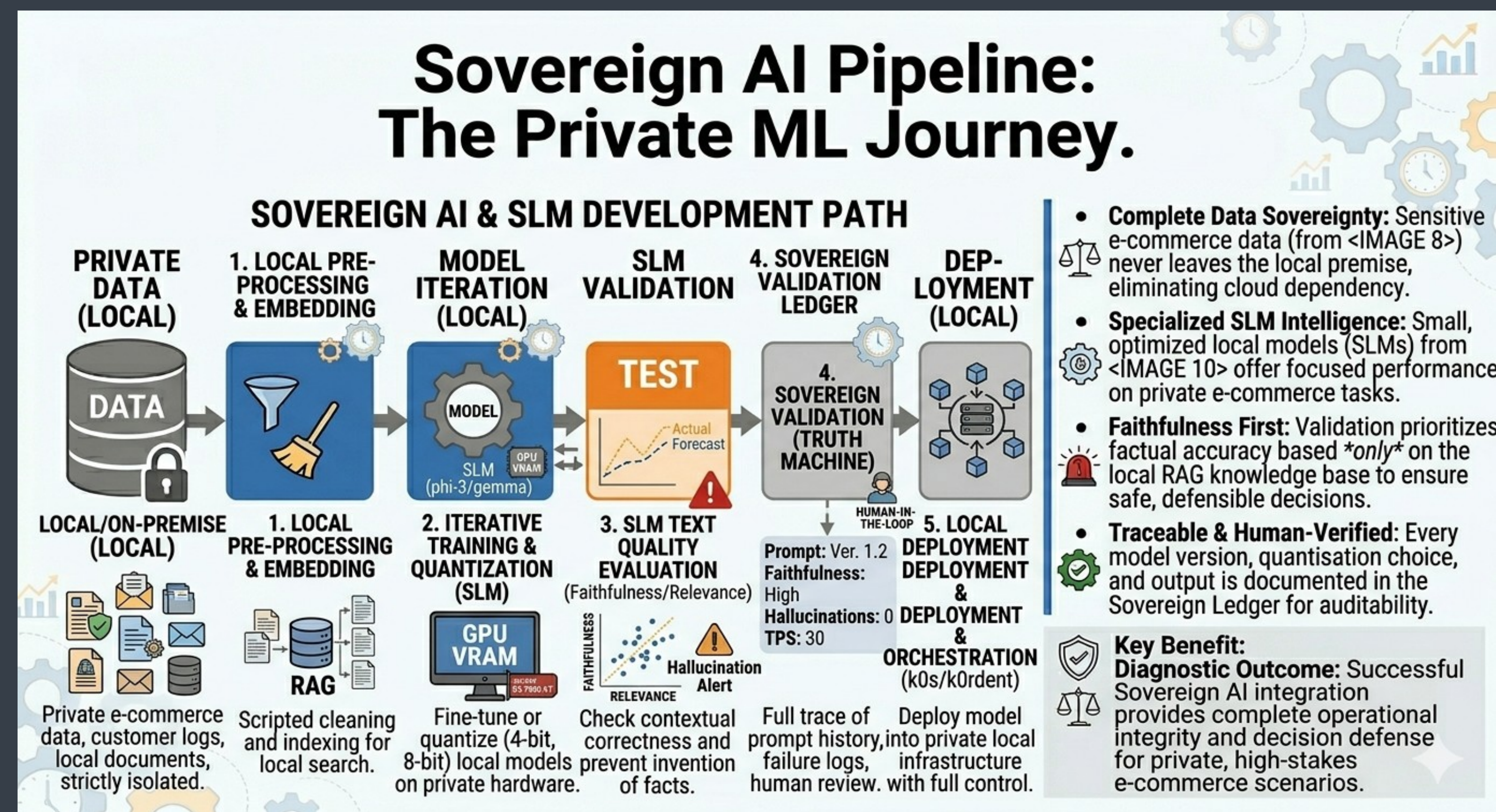
- Directional Accuracy: Check if the model predicts the "Up/Down" trend correctly.
- Document the 'Why': Explain the logic behind your feature selection in the Ledger.



The "Sovereign AI" Angle: Validating Local Models

We have been moving from "Giant Cloud Models" (ie. ChatGPT) to "Local, Specialized Intelligence." (ie. Qwen or Ollama trained on your data).

What is Sovereign AI? It is the ability of an organization to own, host, and control its AI infrastructure and data without relying on third-party cloud providers.

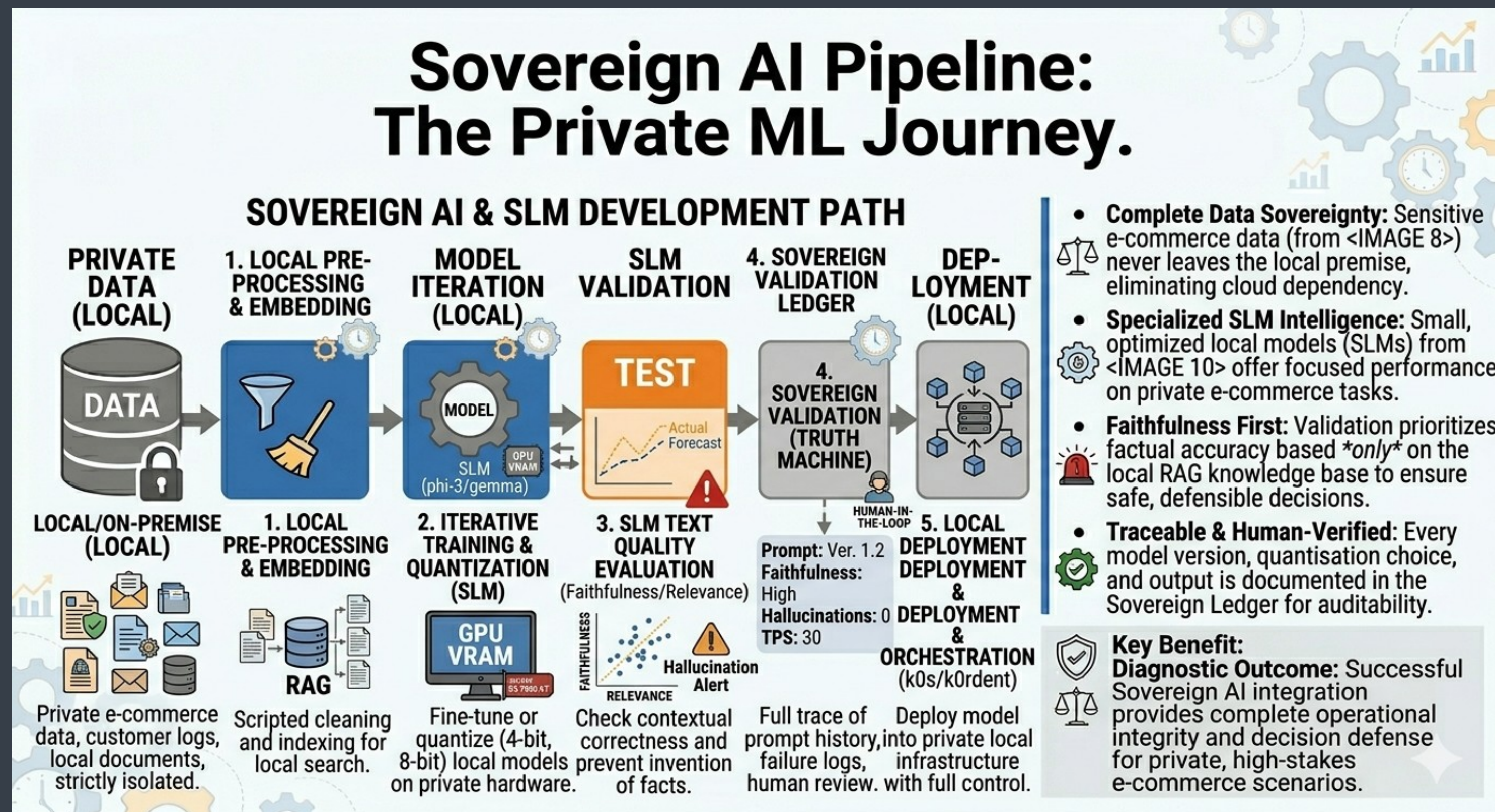


The "Sovereign AI" Angle: Validating Local Models

The Trend (SLMs): Instead of using massive models (LLMs), we now use Small Language Models (SLMs) like Phi-3, Gemma, or Mistral.

Why SLMs?

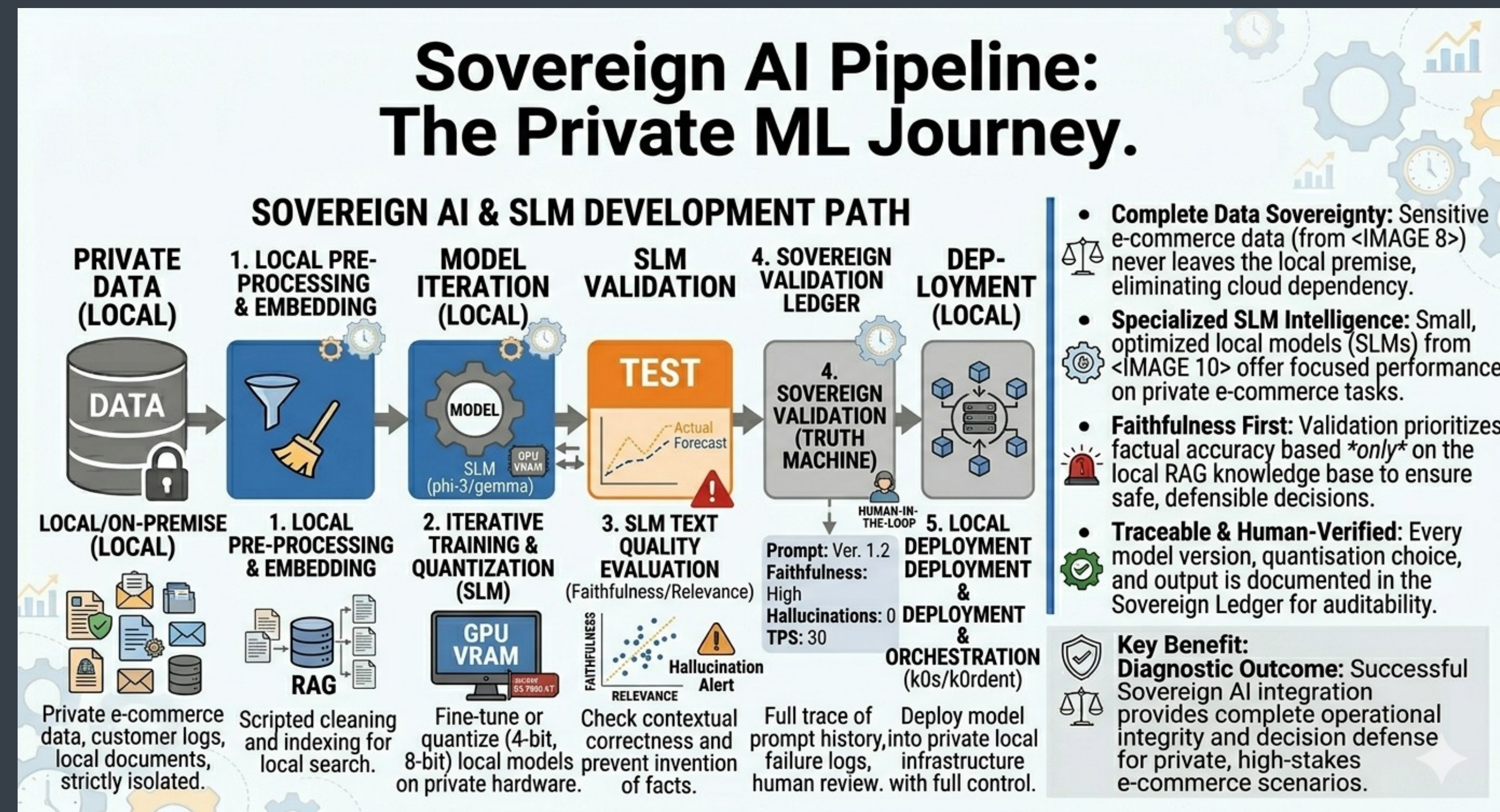
- Efficiency: They can run on local GPUs (like your RX 7900 XT) or even high-end laptops.
- Privacy: Sensitive data never leaves your local server.
- Cost: No API fees; you pay only for electricity and hardware.



The "Sovereign AI" Angle: Validating Local Models

Key takeaway: Smaller, specialized models often outperform giants in specific, private tasks.

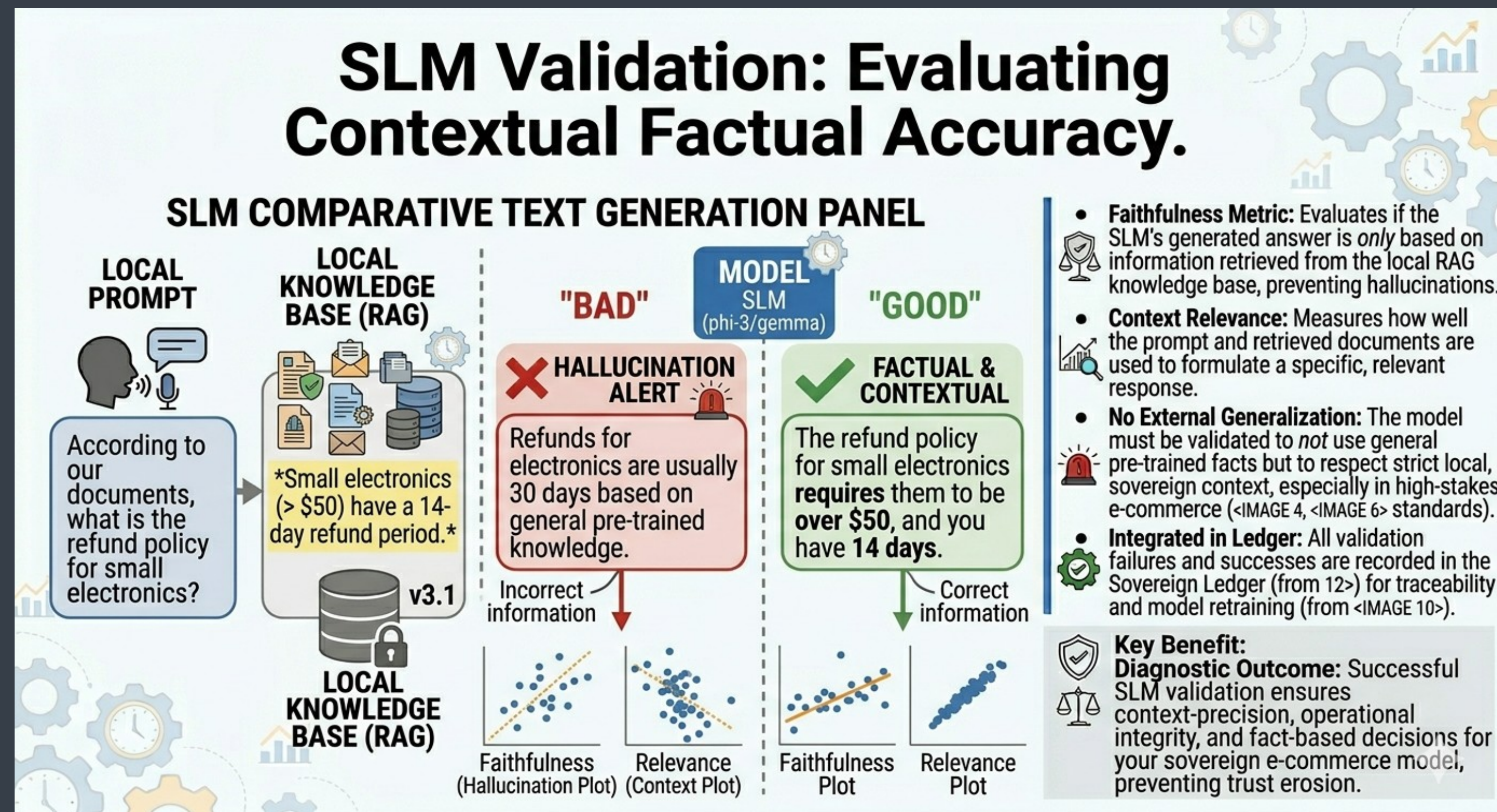
But then, isn't this just ML? Where do local models fit into the "Iterative Journey of Machine Learning"?



The "Sovereign AI" Angle: Validating Local Models

Data Prep (Local): Cleaning and labeling your private documents for RAG (Retrieval-Augmented Generation) or Fine-tuning.

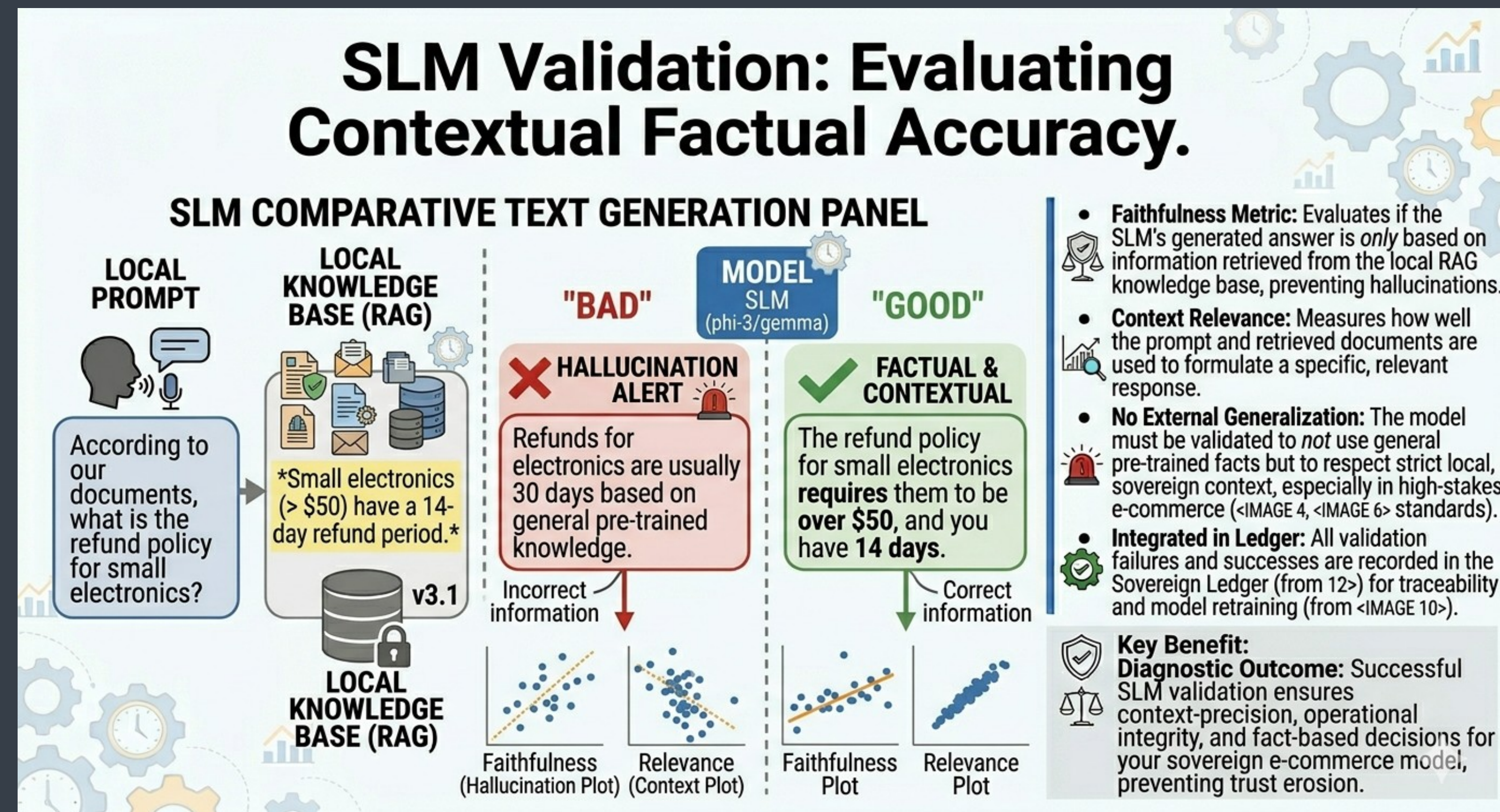
Validation Step: The core of the pipeline. How do we test a local model? It's not just about MSE anymore; it's about "Text Quality" and "Instruction Following."



The "Sovereign AI" Angle: Validating Local Models

Deployment: The model moves from your development machine to a local "k0s" or "k0rdent" orchestrated cluster.

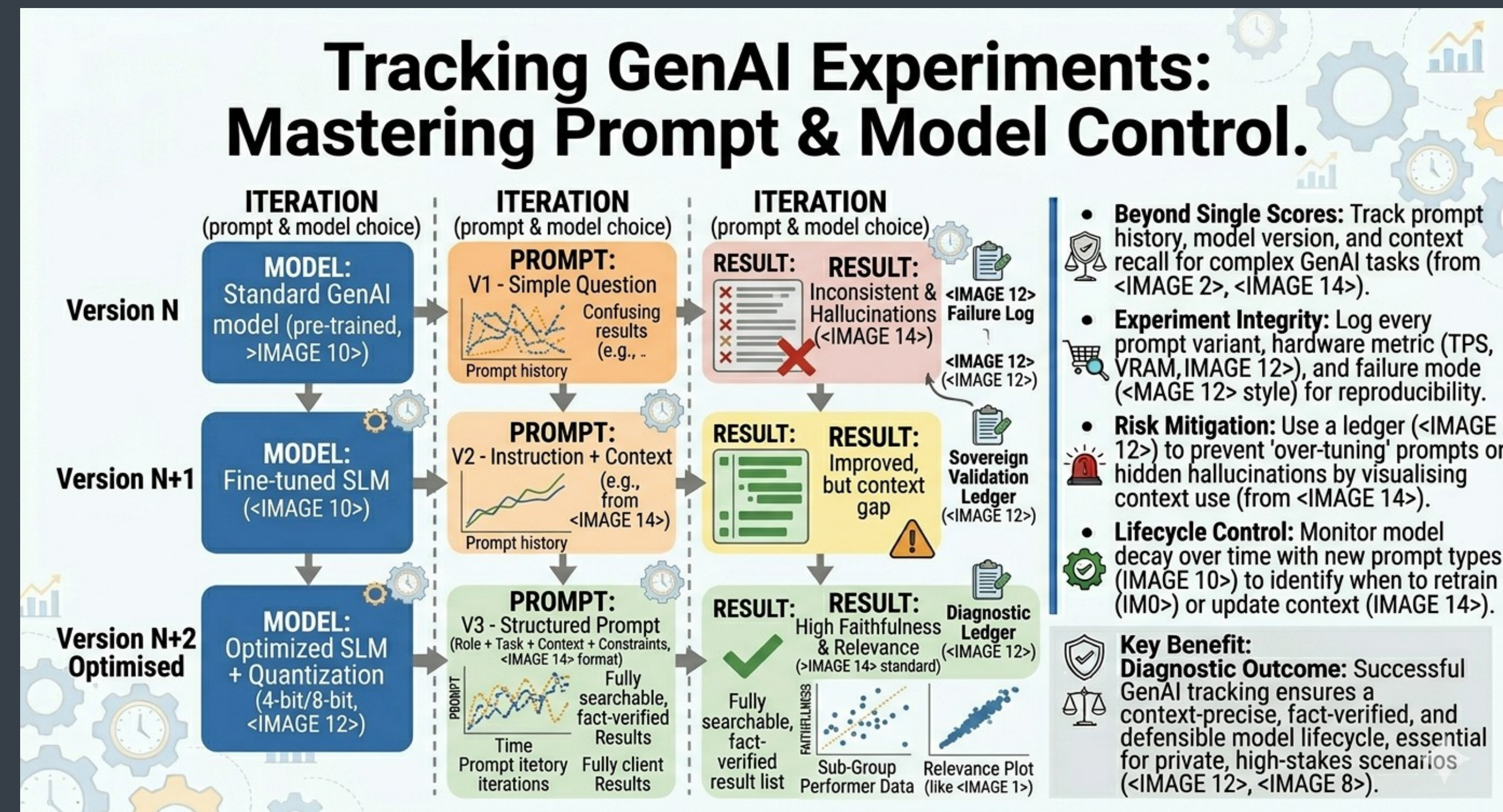
The Difference: In standard ML, you predict a number. In Sovereign AI, you generate a reliable, fact-based response.



The "Sovereign AI" Angle: Validating Local Models

Tracking experiments in the world of Generative AI.

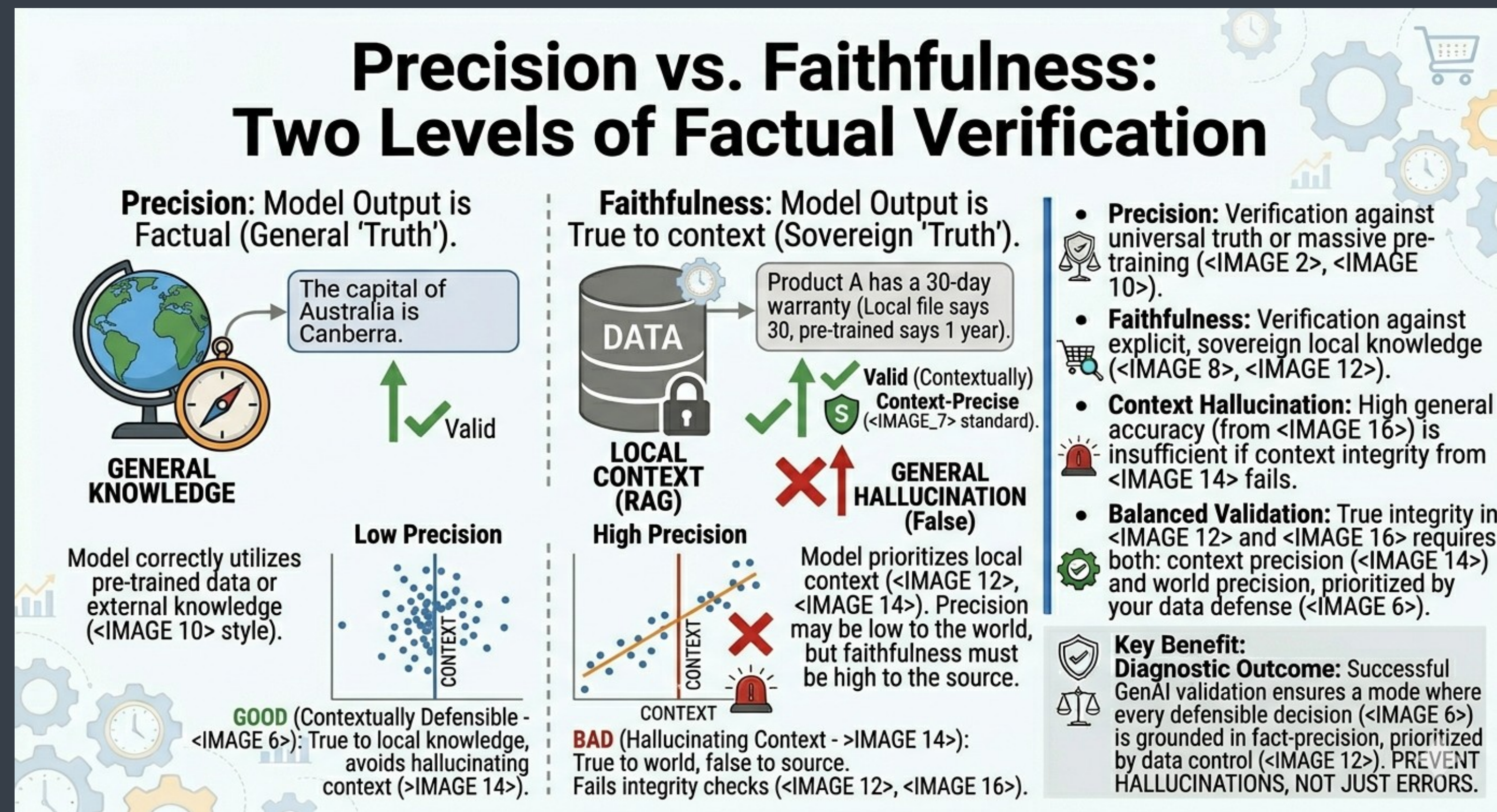
- Prompt Engineering Logs: Recording which "System Prompt" worked best for your task.
- Model Versioning: Tracking different versions (quantization levels like 4-bit, 8-bit) and their impact on speed vs. accuracy.



The "Sovereign AI" Angle: Validating Local Models

How to measure if your local model is actually "good."

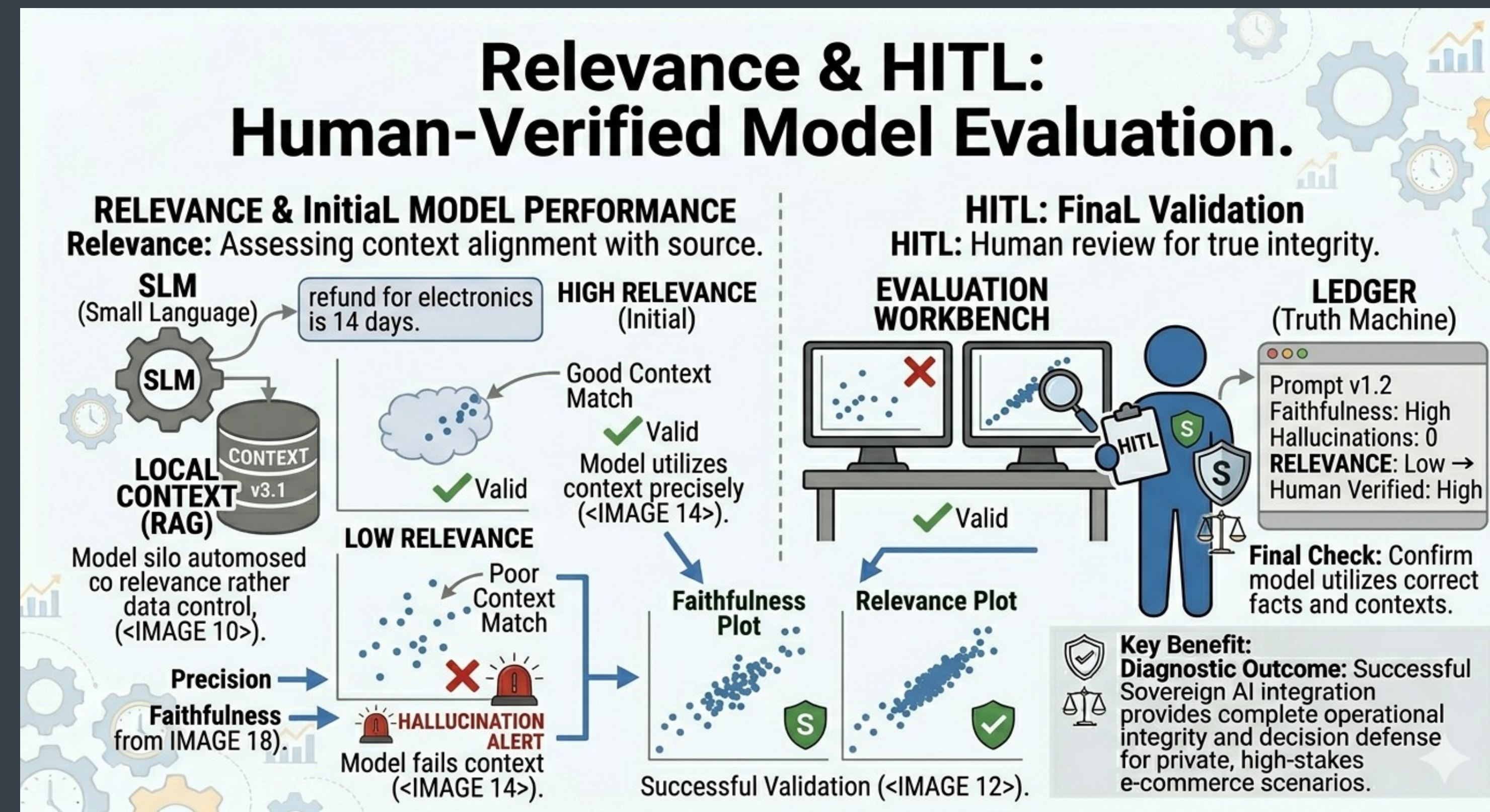
- Context Precision: Did the model find the correct information in your private documents?
- Faithfulness: Did the model answer based only on your data, or did it use "pre-trained knowledge" (hallucination)?



The "Sovereign AI" Angle: Validating Local Models

How to measure if your local model is actually "good." (cnt'd)

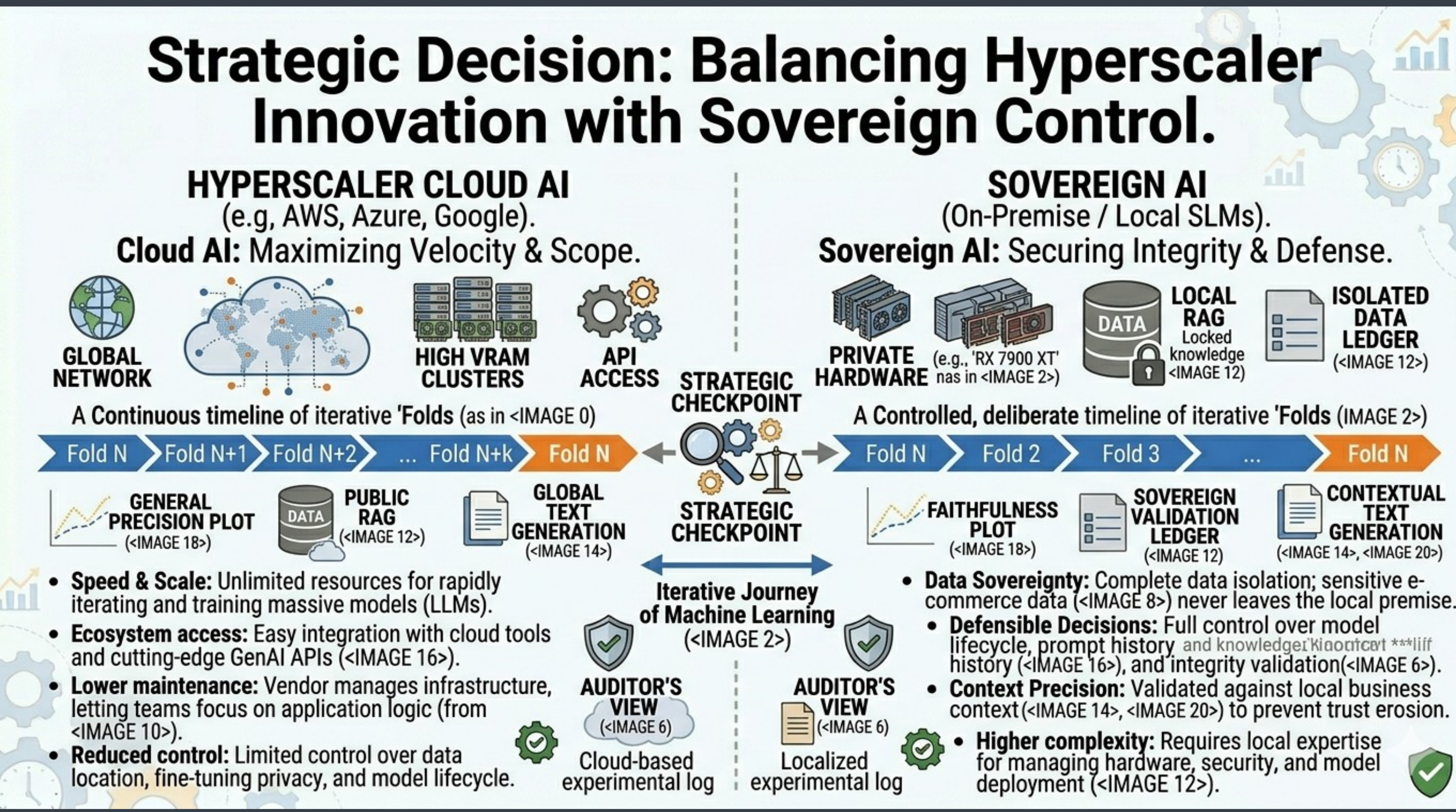
- Answer Relevance: Does the answer solve the user's specific problem?
- The "Human-in-the-loop" Ledger: A final check where an expert (you) scores the model's responses and records them in the validation ledger.



The "Sovereign AI" Angle: Validating Local Models

Strategic Decision: Cloud vs. Sovereign AI:
You may need to defend the choice of local infrastructure.

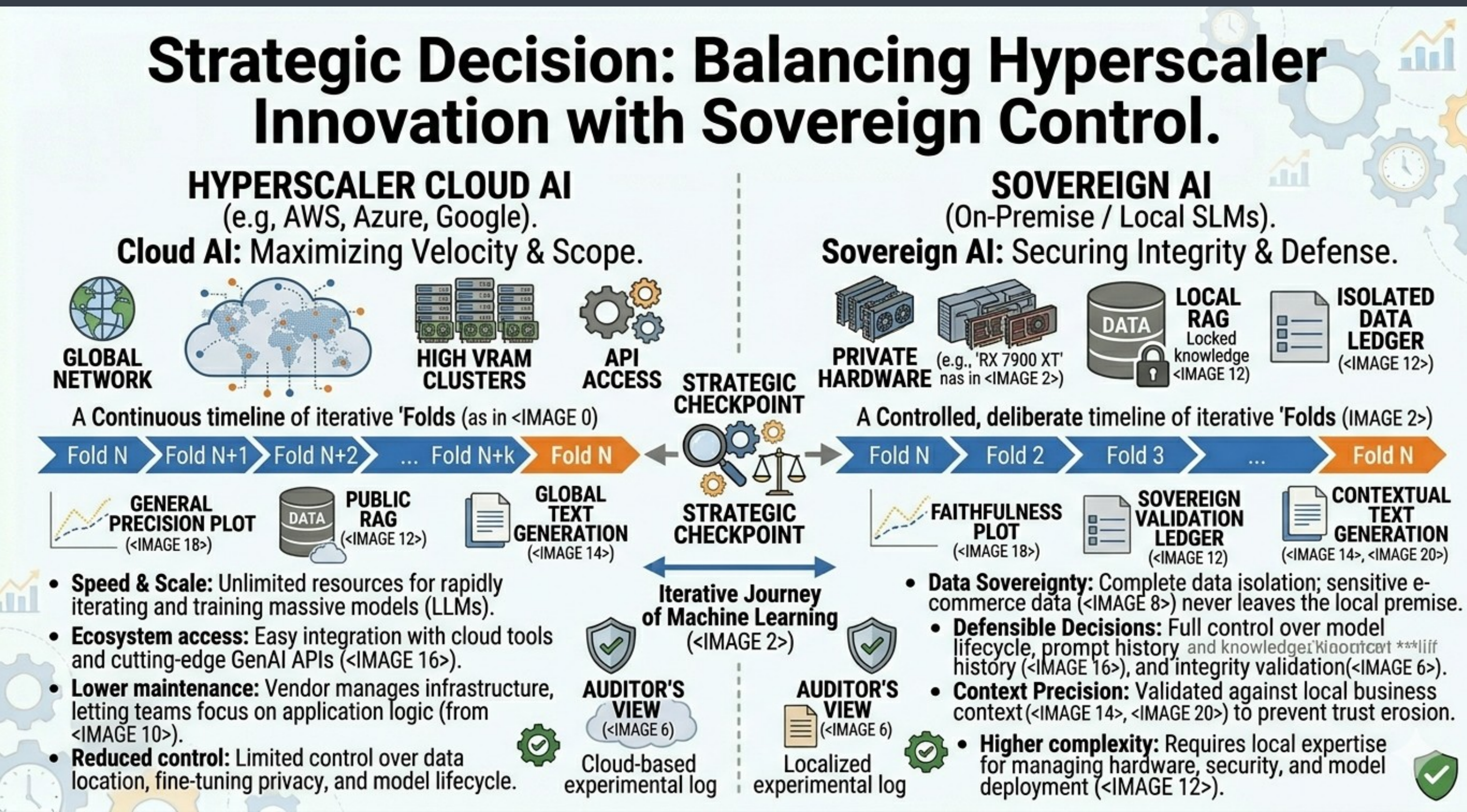
Decision Defense: "We chose an SLM because our data is too sensitive for the cloud and we need 100% availability without internet."



The "Sovereign AI" Angle: Validating Local Models

Operational Integrity: By validating locally, we ensure that our "Decision Defense" is based on data we fully control.

Future-Proofing: Learning to manage local SLMs today prepares you for the next era of private enterprise AI.

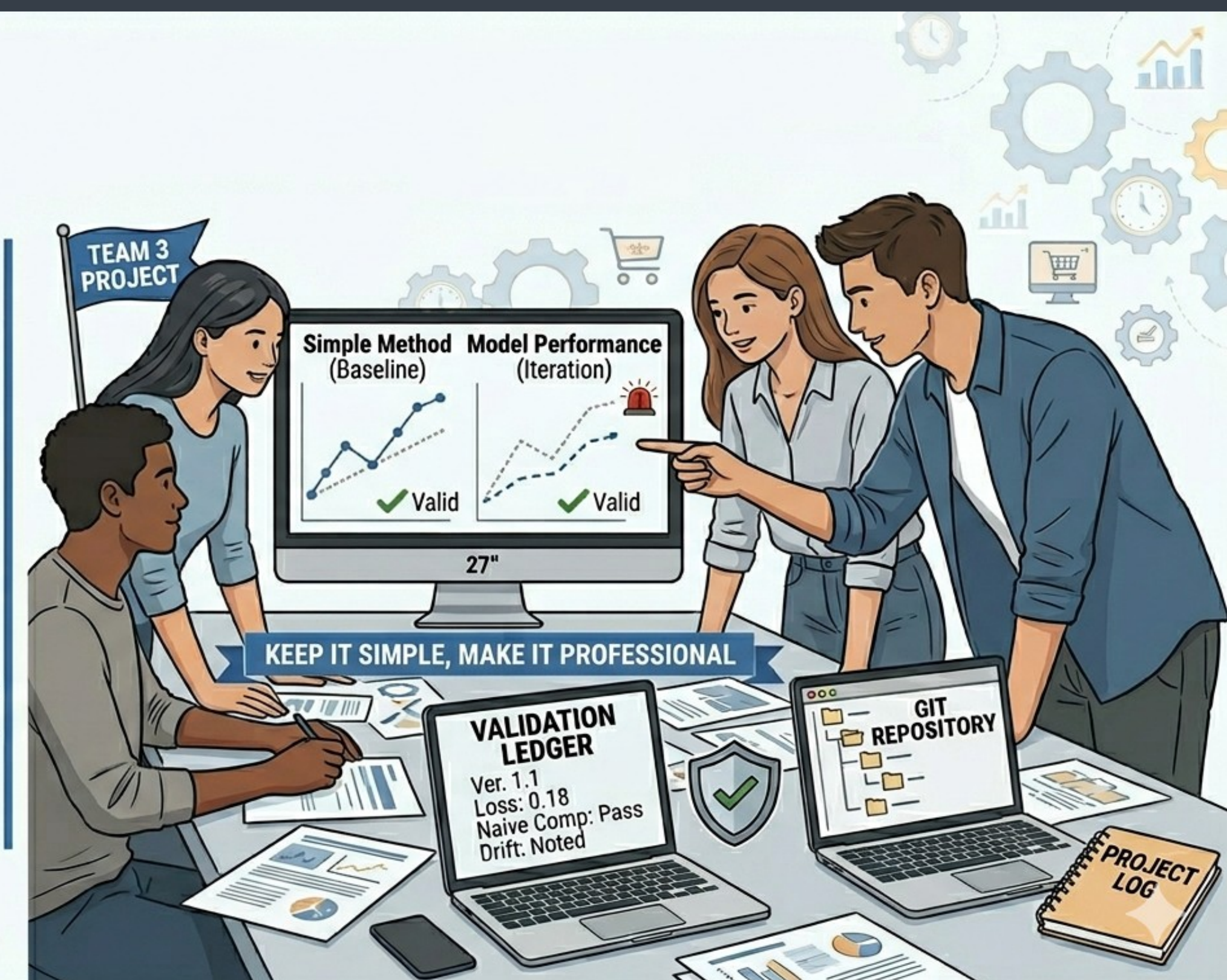
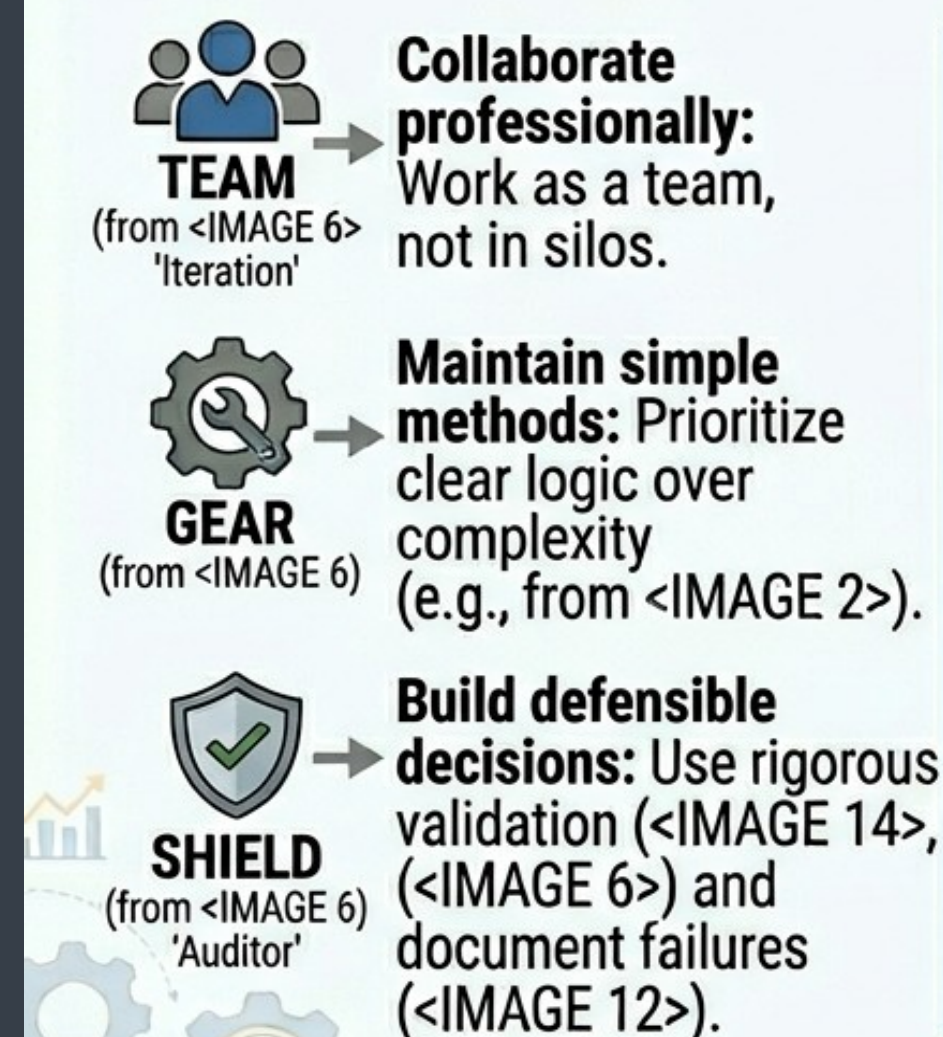


Term Project Roadmap & "Decision Defense"

Focus on the process, not just the code.

Step 1: Form Your Team: Find 2-3 partners who share your commitment. Diversity in skills (coding, data cleaning, documentation) is an advantage.

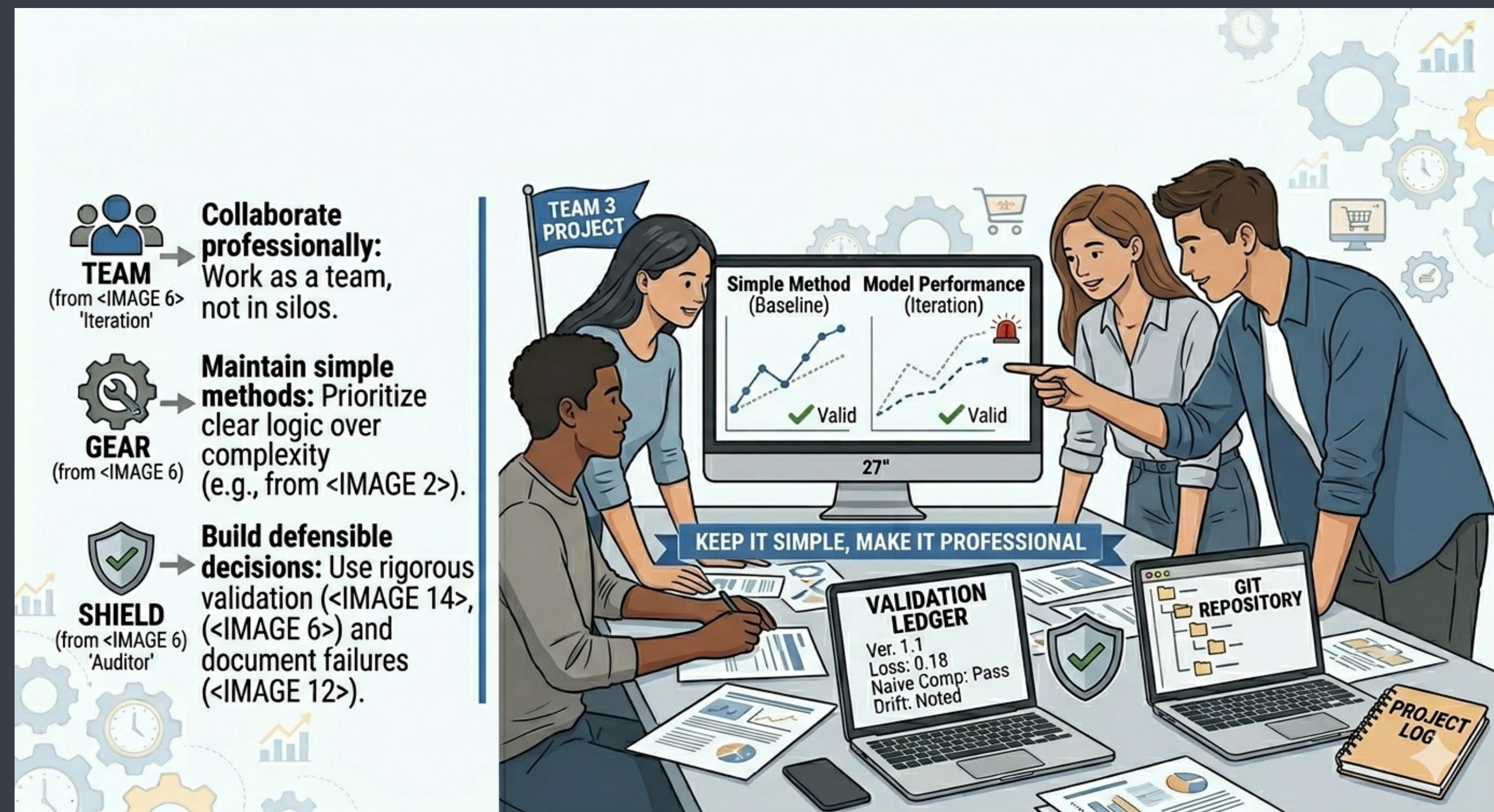
Step 2: Choose a "Simple" Problem: Do not try to solve world hunger. Pick a clear, well-defined problem with accessible data.



Term Project Roadmap & "Decision Defense"

Step 3: Build the Pipeline: Apply the "Discipline of Discovery." Every step must be reproducible.

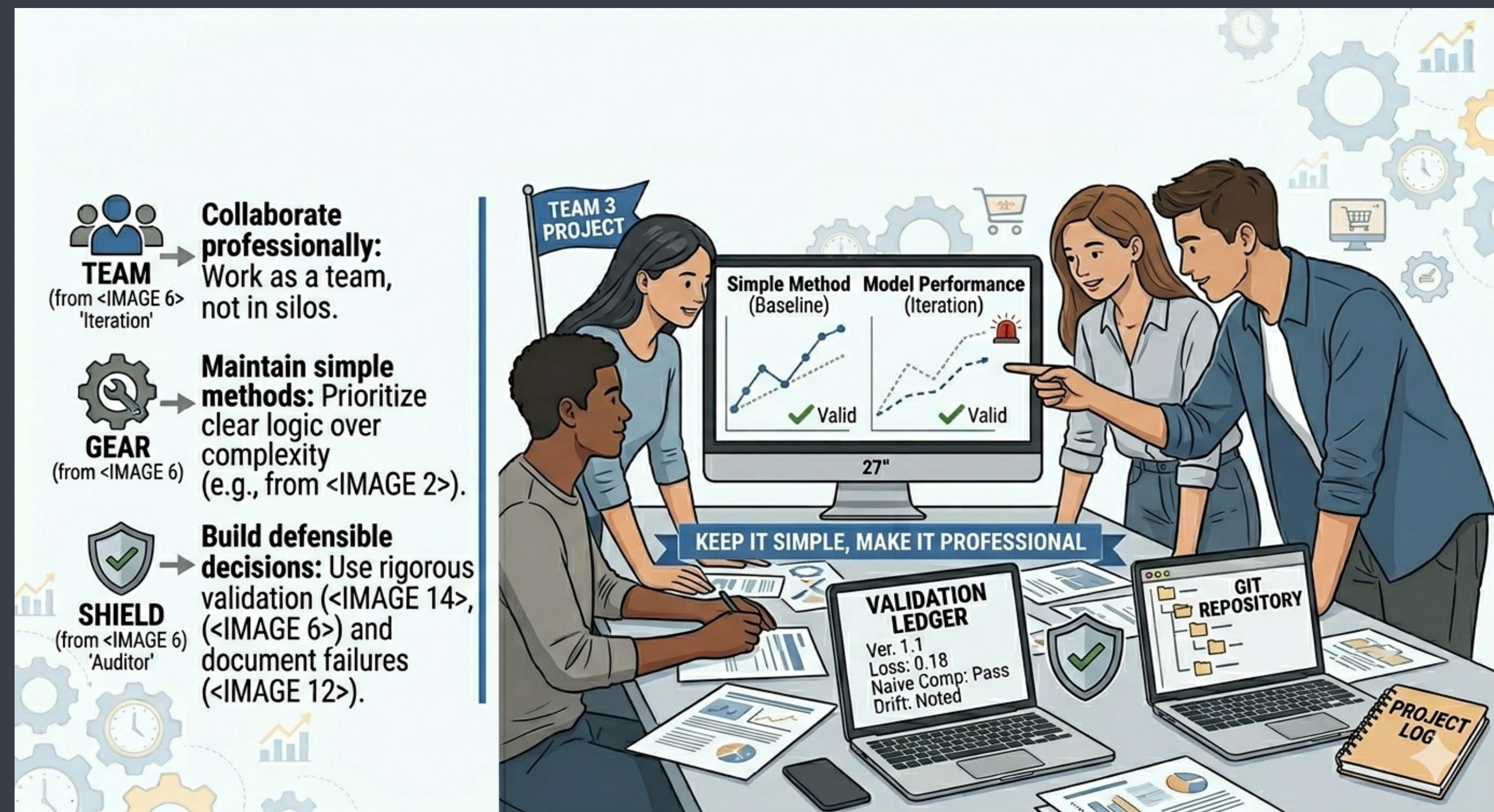
Step 4: The Final Defense: You will not just show an accuracy score; you will defend your "Decision Integrity."



Term Project Roadmap & "Decision Defense"

Complexity is the enemy of reliability.

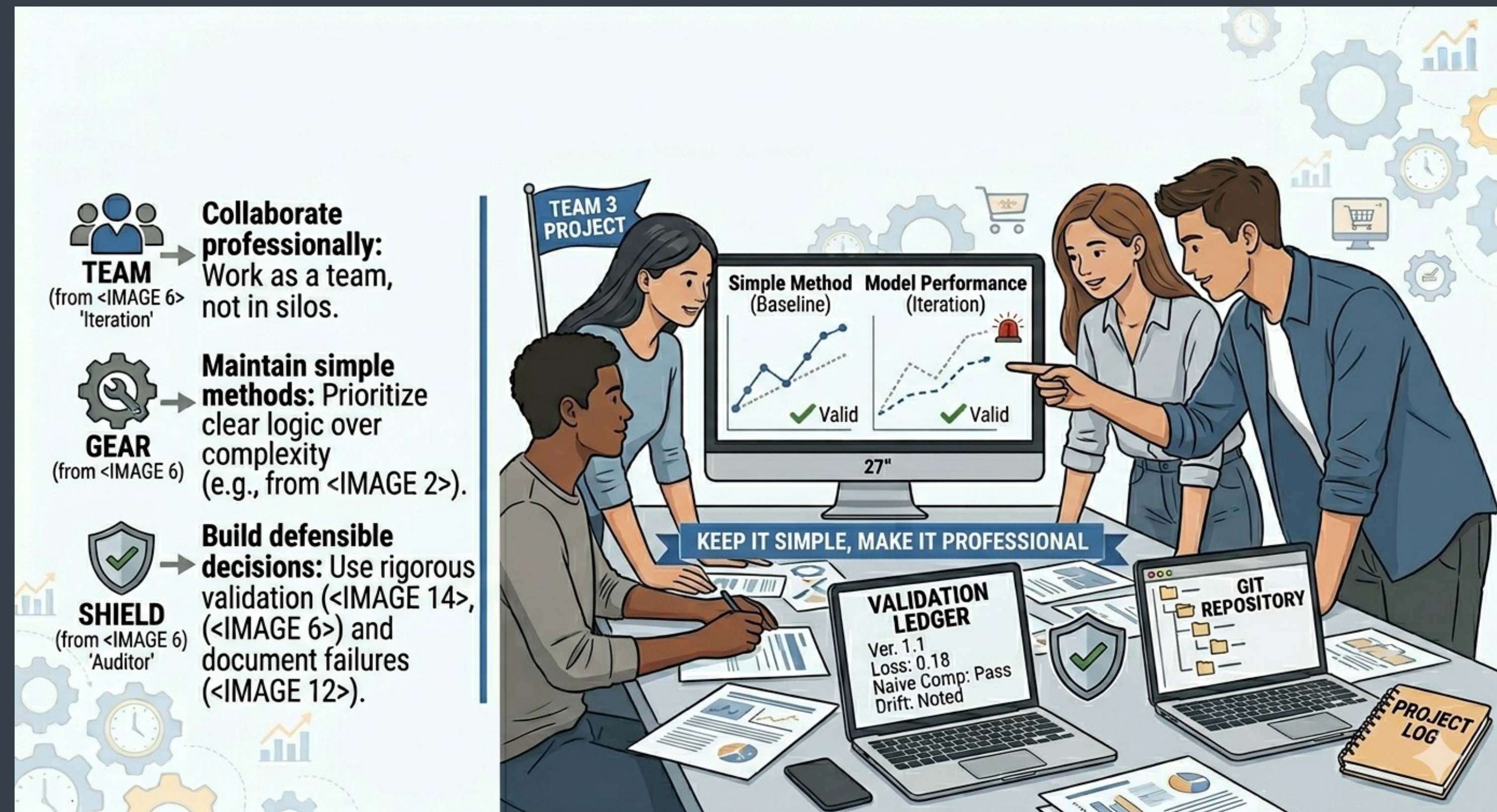
Simple Method > Complex Chaos: A well-explained Linear Regression with a perfect "Validation Ledger" is worth more than a "Black Box" Neural Network that you cannot explain.



Term Project Roadmap & "Decision Defense"

The "Professional" Difference:

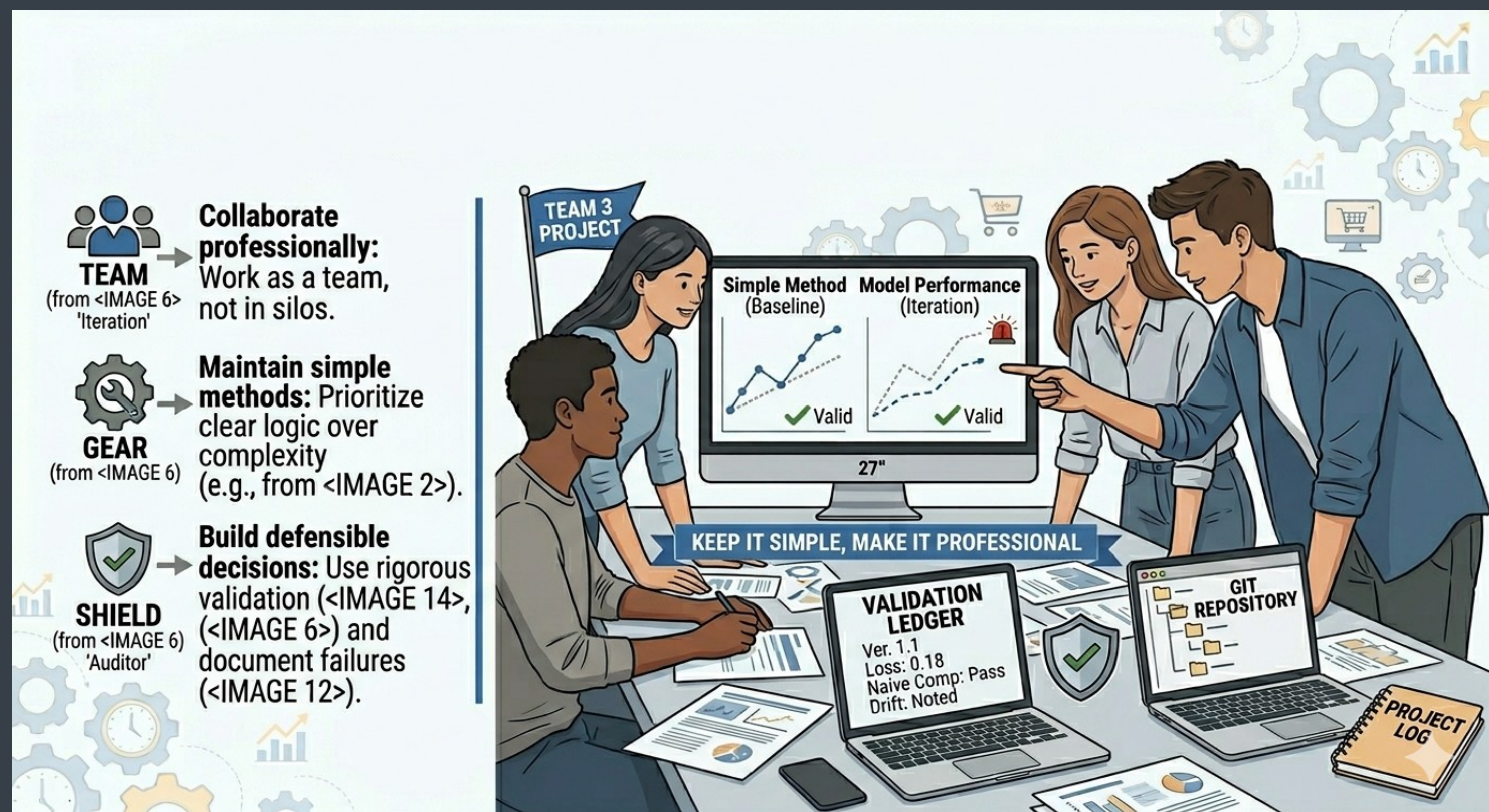
- Clean Code: Use meaningful variable names and comments.
- Version Control: Use GitHub/GitLab as if you are in a high-tech company.
- Automation: Your pre-processing and validation should run with a single script.
- Key takeaway: We are looking for "Engineered Solutions," not "Lucky Guesses."



Term Project Roadmap & "Decision Defense"

In the real world, ML is a team sport.

- Collaboration: Use professional tools for communication and task management. Don't work in silos; review each other's code.
- The "Ledger" as a Team Hub: Your experiment log should be accessible to all members. Anyone in the team should be able to explain any result.

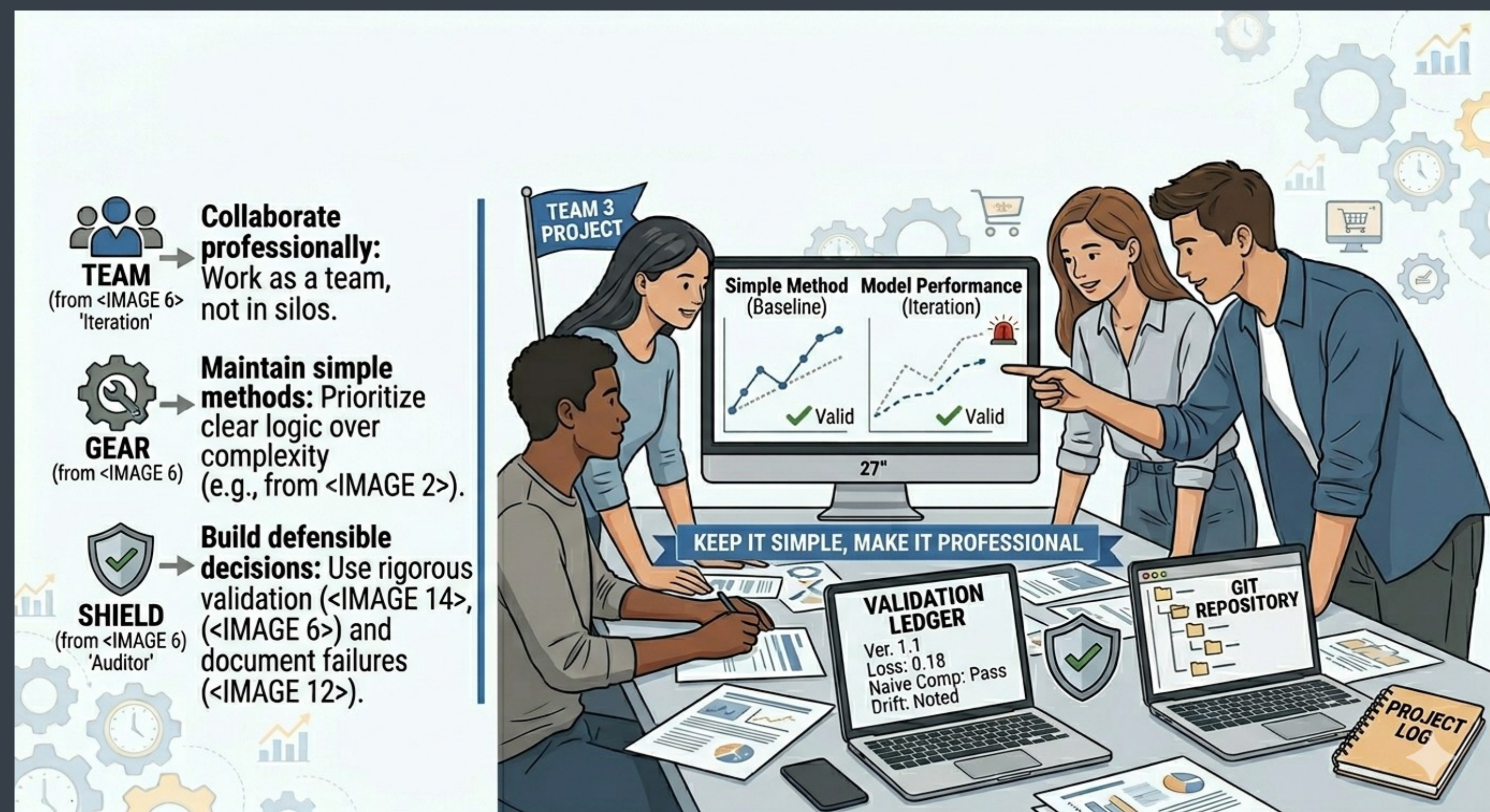


Term Project Roadmap & "Decision Defense"

In the real world, ML is a team sport.

(cnt'd)

- Consistency: Ensure your data cleaning logic and evaluation metrics are the same across all experiments.
- Accountability: If the model fails (high residuals/drift), work together to find the "Why" instead of blaming the data.



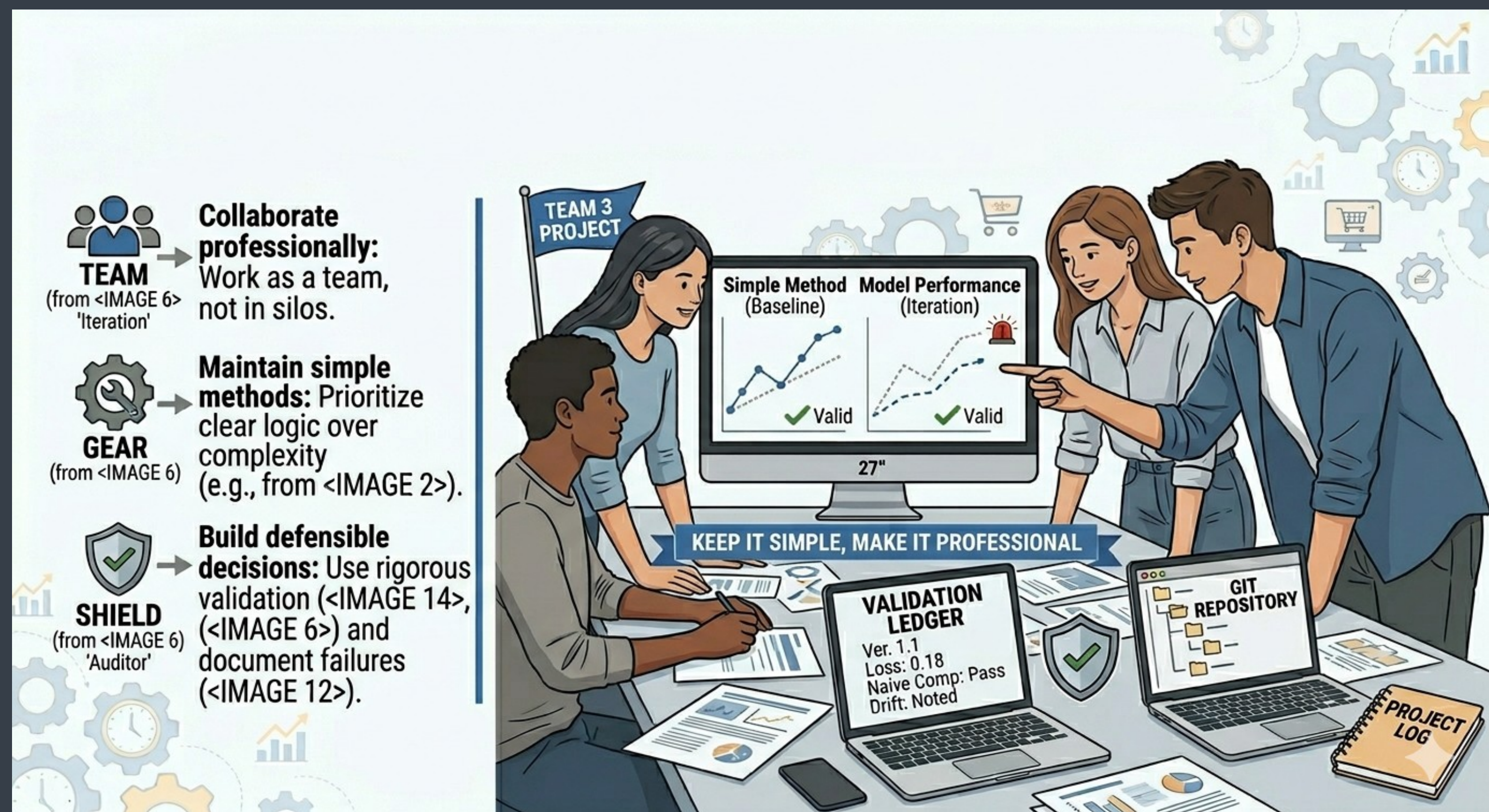
Term Project Roadmap & "Decision Defense"

Preparing for the final presentation.

- It's Not About the Score: We don't care if your accuracy is 99% if you cannot prove your validation was honest.

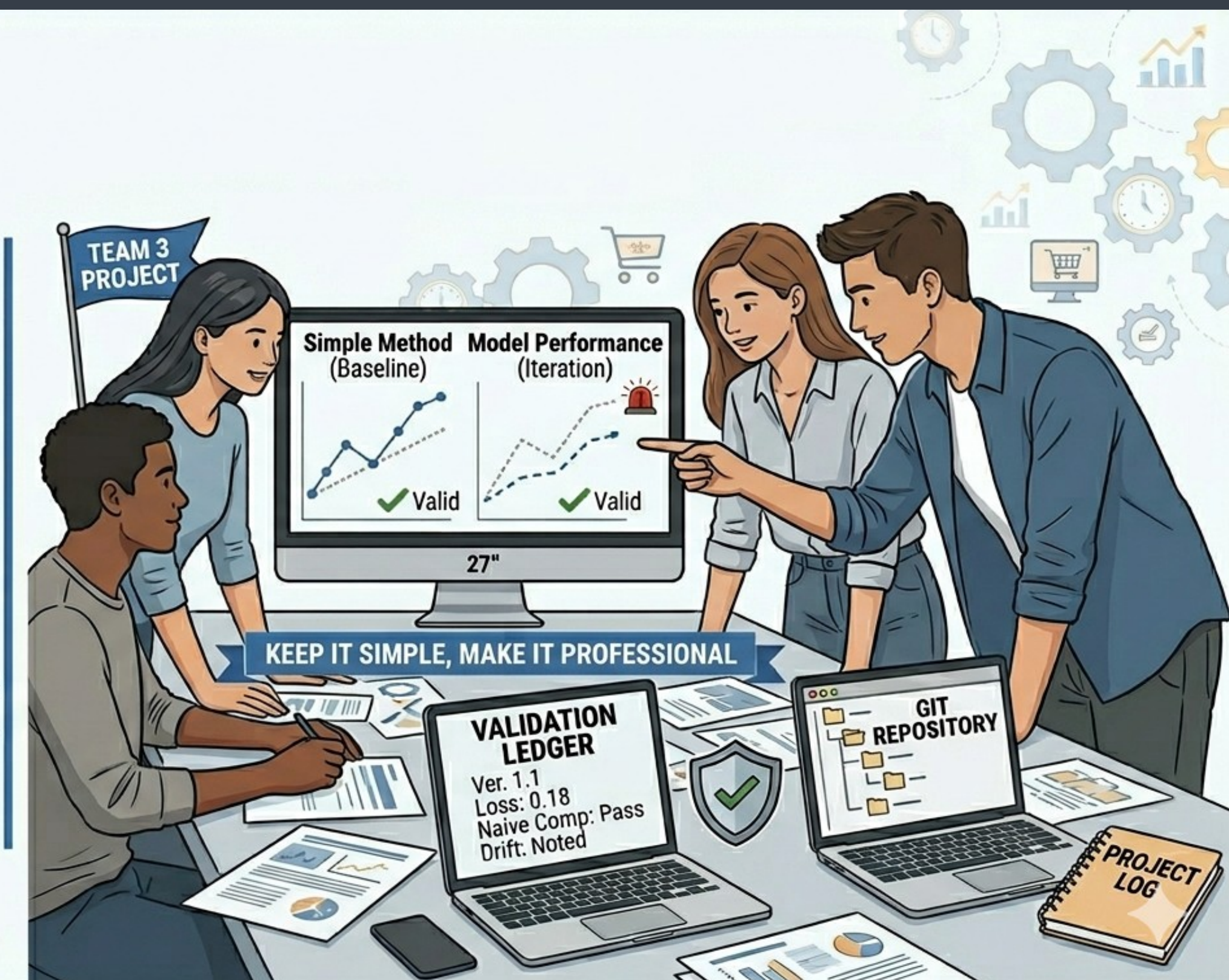
The Defense Checklist:

- Did you check for Residual Patterns?
- Did you compare against a Naive Forecast?
- Is your "Sovereign" Data handled with integrity?
- Can you explain your Failures as clearly as your successes?



Term Project Roadmap & "Decision Defense"

The Goal: To convince a "Manager" (the Instructor) that your model is safe and reliable for a real business decision.

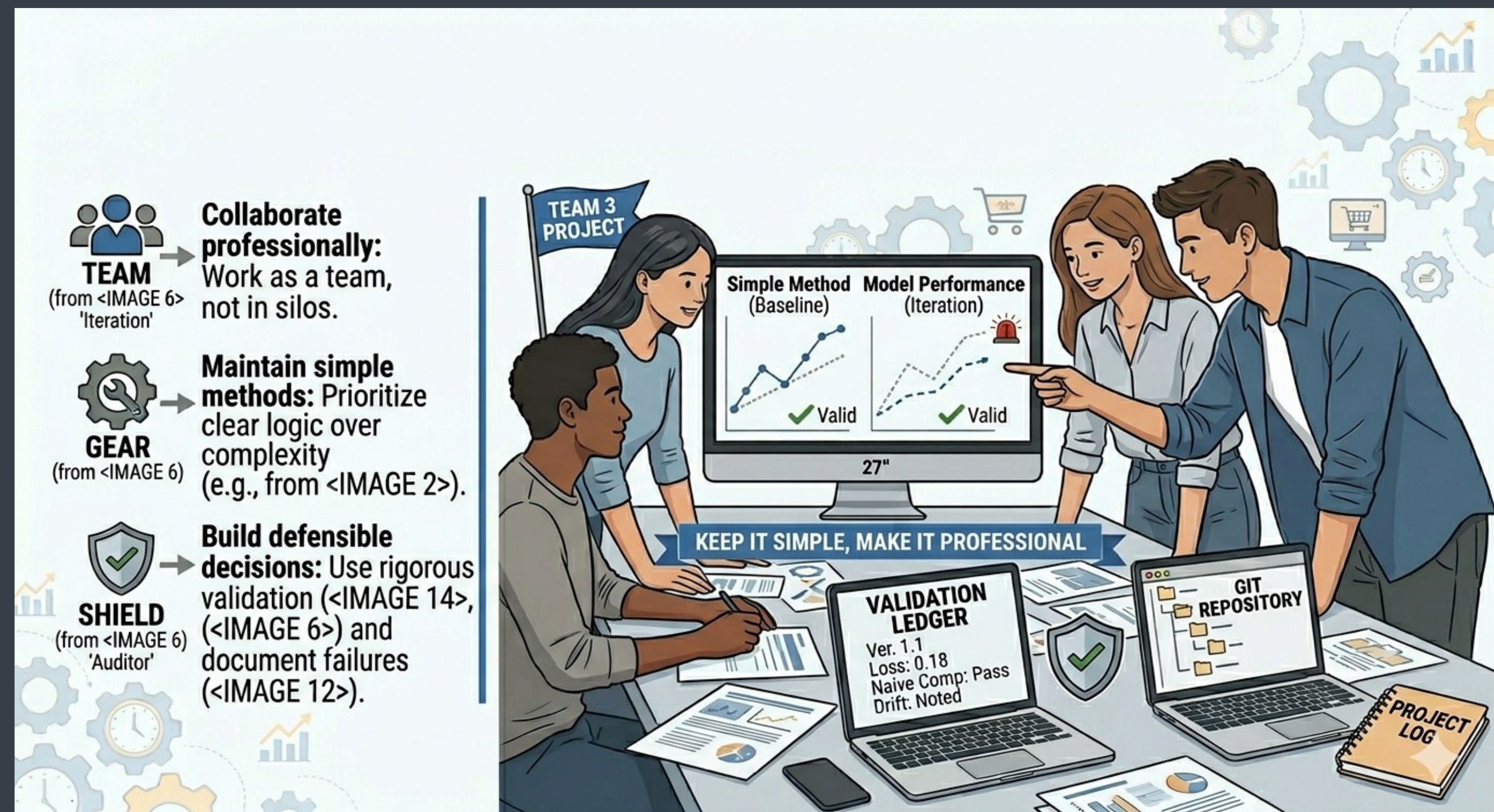


Term Project Roadmap & "Decision Defense"

Project Milestones (Timeline)

Stay on track and avoid "Last-Minute" disasters.

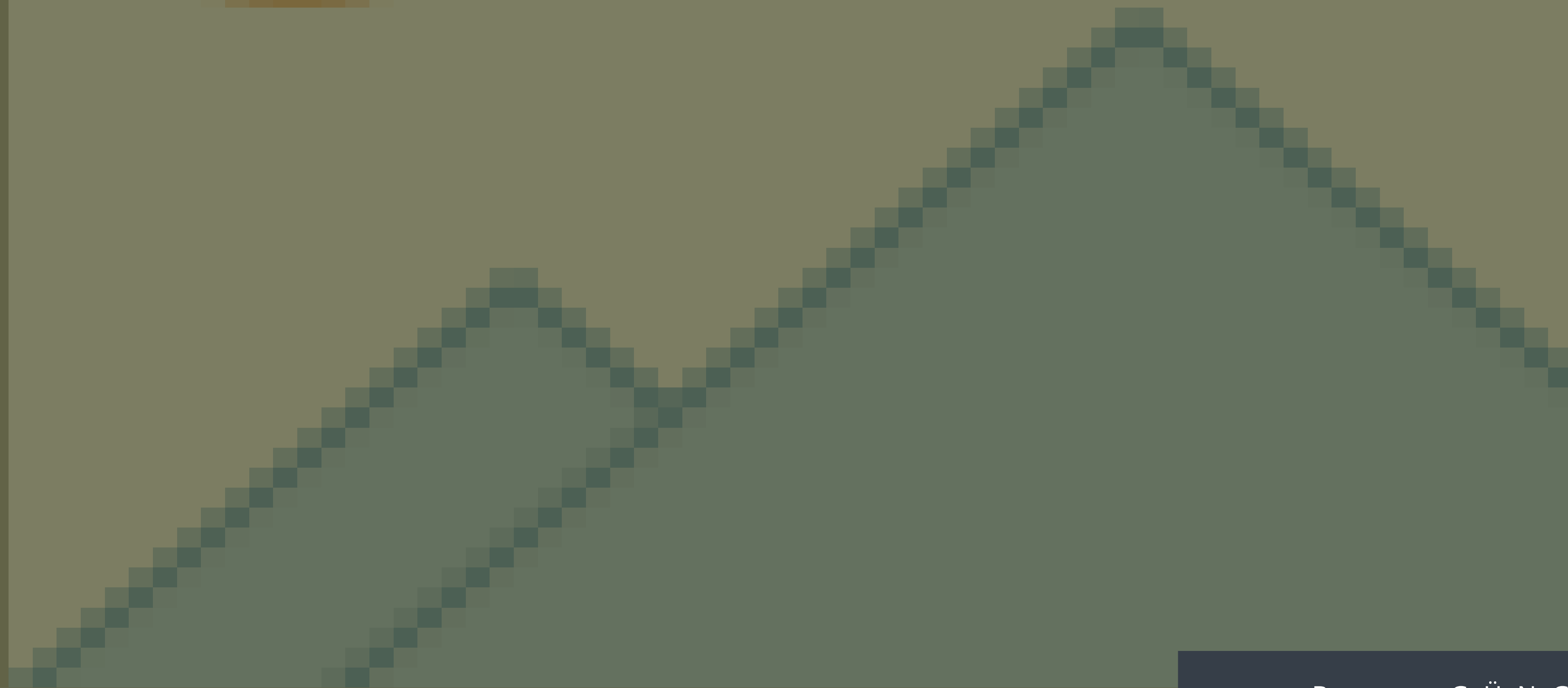
- Team formation and Problem Definition (The "Simple" Choice).
- Data Collection and initial Pipeline setup (The "Notebook" Phase).
- Iterative Modeling and Ledger Documentation (The "Discipline" Phase).
- "Decision Defense" Presentations.



Thank You



Please follow up on Github and your emails (school emails).



B o r a G Ü N G Ö R E N